



北京邮电大学世纪学院
Century College, Beijing University of Posts and Telecommunications

课程 设计 报 告

课程设计名称 企业实践

专 业 软件工程

班 级 1 班

学 号 24160131

姓 名 王可昕

指导教师 侯腾飞

成 绩

2026 年 7 月 5 日

目 录

1.前言	1
1.1 课题背景与意义	1
1.2 本课题主要工作	1
1.3 开发环境	2
2.需求分析	3
2.1 需求分析概述	3
2.2 需求分析内容	3
3.概要设计	5
3.1 系统框架结构	5
3.2 系统功能结构图	6
3.3 系统 E-R 图	7
3.4 数据库设计	8
3.4.1 用户表 user	8
3.4.2 文章表 post	8
3.4.3 友链表 friend_link	9
3.4.4 配置表 config	10
3.4.5 邮件日志表 email_log	10
3.4.6 邮箱验证码表 email_verification	11
4.系统详细设计	11
4.1 小程序首页模块设计	11
4.2 文章详情模块设计	12
4.3 归档模块设计	13
4.4 后台登录认证模块设计	14
4.5 后台文章管理模块设计	15
4.6 友链管理与展示模块设计	16
4.7 系统配置模块设计	17
4.8 图片上传模块设计	18
5.系统实现	19
5.1 小程序文章详情阅读功能	19

5.2 小程序接口请求与文章列表功能.....	25
5.3 后台文章封面上传功能.....	29
5.4 文章初始化与课程笔记数据写入功能.....	33
5.5 友链展示与技术资料入口功能.....	39
5.6 系统配置与个性化信息功能.....	42
6.系统测试.....	46
6.1 系统测试概述.....	46
6.2 测试用例.....	48
6.2.1 小程序首页模块测试.....	48
6.2.2 文章详情模块测试.....	50
6.2.3 归档模块测试.....	52
6.2.4 友链模块测试.....	54
6.3 系统测试总结.....	56
7.总结体会.....	57
8.参考资料.....	58

博客小程序

1. 前言

1.1 课题背景与意义

随着移动互联网和前端技术的快速发展,用户获取知识和浏览内容的方式逐渐从传统网页转向移动端、小程序端等轻量化平台。对于学习类内容而言,传统的笔记整理方式往往存在内容分散、阅读体验不统一、检索不方便等问题,难以满足用户在碎片化时间内进行复习、查阅和知识回顾的需求。因此,设计并实现一个面向移动端阅读场景的博客小程序,具有一定的现实意义和应用价值。

本课题以 Vue 3 课程学习内容为基础,设计并实现一个集文章展示、章节归档、技术链接展示和后台内容管理于一体的博客系统。系统前台采用 uni-app 开发微信小程序端,方便用户在移动设备上浏览课程笔记;后台管理端采用 Vue 3 技术栈,用于文章、友链、站点配置等内容的维护;后端使用 Bun、Elysia 和 SQLite 提供数据接口与持久化支持。项目整体采用前后端分离思想,使展示端、管理端和服务端职责清晰,便于后期维护和扩展。

通过本课题的设计与实现,可以将课堂学习内容与实际项目开发相结合。一方面,系统能够把 Vue 3 课程内容整理为结构清晰、适合移动端阅读的章节笔记,提高学习资料的管理效率;另一方面,项目覆盖了前端页面设计、接口请求、后端服务、数据库设计、文件上传、内容管理等多个环节,有助于加深对现代 Web 与小程序开发流程的理解,提升综合实践能力。

1.2 本课题主要工作

本课题主要完成了一个基于前后端分离架构的博客小程序系统。系统围绕“Vue 3 课程笔记展示与管理”这一核心目标展开,实现了小程序前台、后台管理端和后端接口服务三个主要部分。

首先,在小程序前台部分,完成了首页、归档页、友链页和文章详情页的设计与实现。首页用于展示置顶文章和最新发布的课程笔记;归档页按照时间顺序展示所有公开文章,方便用户快速查找不同章节内容;友链页展示与项目技术栈相关的优秀官方网站和学习资源;文章详情页用于完整展示文章内容、封面、标签、阅读量等信息,并针对移动端阅读体验进行了界面优化。

其次,在后台管理端部分,完成了管理员登录、文章管理、友链管理、邮件日志管理、站点配置和个性化信息配置等功能。管理员可以通过后台维护文章标题、正文内容、标签、封面图、发布状态等数据,也可以修改站点标题、头像、首页横幅、社交链接和页脚信息。后台编辑器支持富文本内容编辑和图片上传,提升了内容维护的便利性。

再次，在后端服务部分，完成了用户认证、文章接口、友链接口、配置接口、上传接口和邮件相关接口的开发。后端负责接收前端请求、校验数据、处理业务逻辑并读写 SQLite 数据库。系统通过数据库保存文章、用户、配置、友链和邮件日志等数据，并通过文件上传模块保存头像、站点图标、文章封面和正文图片等资源。

最后，在系统测试与优化方面，对小程序前台页面跳转、文章详情展示、后台保存、文件上传、数据库初始化、真机调试访问等功能进行了调试和完善。针对小程序端 UI 效果较弱、页面跳转割裂、真机无法访问本地后端、初始化数据覆盖后台修改等问题进行了分析和改进，使系统在功能完整性、界面美观性和使用稳定性方面得到提升。

1.3 开发环境

本系统采用前后端分离开发方式，整体项目由小程序前台、后台管理端和后端服务三部分组成。开发过程中使用 Windows 操作系统作为主要开发环境，使用 Visual Studio Code 编写代码，使用微信开发者工具进行小程序预览与真机调试，使用浏览器调试后台管理端页面。

系统前台使用 uni-app 和 Vue 3 开发，并编译为微信小程序运行。uni-app 具有跨端开发能力，能够使用 Vue 语法组织页面结构和业务逻辑，适合开发移动端小程序界面。后台管理端使用 Vue 3、Vite、Pinia、Vue Router 等技术实现，负责系统数据的可视化管理和内容维护。后端服务使用 Bun 作为 JavaScript 运行时，采用 Elysia 框架开发接口服务，使用 SQLite 数据库存储系统数据，并通过 Drizzle ORM 管理数据库结构和查询操作。

类别	工具或技术	说明
操作系统	Windows 10 / Windows 11	项目主要开发与调试环境
代码编辑器	Visual Studio Code	编写前端、后端和配置文件
小程序工具	微信开发者工具	小程序预览、调试和真机测试
前台框架	uni-app、Vue 3	实现微信小程序前台页面
后台框架	Vue 3、Vite、Pinia、Vue Router	实现后台管理端页面和状态管理
后端运行时	Bun	运行后端服务和脚本
后端框架	Elysia	提供 REST API 接口服务
数据库	SQLite	保存文章、配置、友链、用户等数据
ORM 工具	Drizzle ORM	管理数据库表结构和数据访问
开发语言	TypeScript、JavaScript、Vue、CSS	系统主要开发语言
接口调试	浏览器、微信开发者工具控制台	调试接口请求和页面数据展示

在项目运行时，通常需要分别启动后端服务和前端开发服务。后端服务用于提供文章、友链、配置和上传等接口；小程序前台通过接口地址访问后端数据；后台管理端通过接口完成登录、内容保存和系统配置维护。通过该开发环境，可以较完整地完成系统的编码、调试、测试和运行验证。

2. 需求分析

2.1 需求分析概述

需求分析是系统设计与实现的基础，其主要目的是在系统开发前明确系统需要完成的功能、面向的用户对象、业务流程以及运行约束。本系统的建设目标是实现一个面向 Vue 3 课程笔记展示与管理的博客小程序平台，使用户能够通过微信小程序便捷地浏览课程章节笔记、查看文章归档和访问项目技术栈相关的官方资料，同时使管理员能够通过后台管理端对文章、友链、站点配置和系统资源进行维护。

从用户角度来看，系统主要服务于两类用户：一类是普通访问用户，主要通过小程序端浏览公开文章、查看归档内容和访问技术链接；另一类是系统管理员，主要通过后台管理端完成文章发布、内容修改、图片上传、友链管理、邮件日志查看和站点配置维护等操作。两类用户的需求不同，因此系统在设计时需要分别考虑前台展示体验和后台管理效率。

从业务角度来看，系统的核心业务围绕“内容发布与内容浏览”展开。管理员在后台创建或维护文章，将 Vue 3 课程内容整理为章节笔记，并设置文章标题、摘要、标签、封面、正文和发布状态。系统将已发布文章提供给小程序端展示，普通用户可以在首页查看推荐内容，在归档页按时间浏览全部公开文章，在文章详情页阅读完整内容，在友链页访问与项目相关的技术资料入口。

从系统架构角度来看，本系统采用前后端分离模式。小程序前台负责移动端页面展示，后台管理端负责系统数据维护，后端服务负责接口提供、业务处理和数据库访问。该设计能够降低各部分之间的耦合度，使前台展示、后台管理和数据服务相互独立，便于后期维护、功能扩展和问题定位。

因此，本系统需求分析不仅要关注文章展示、文章管理、友链展示、配置管理等功能性需求，还需要考虑系统的可用性、稳定性、安全性、可维护性和移动端适配效果。通过对需求进行分析，可以为后续的概要设计、数据库设计、详细设计和系统实现提供依据。

2.2 需求分析内容

根据系统建设目标和实际使用场景，本系统的需求可以分为普通用户需求、管理员端需求、后端服务需求以及非功能性需求四个方面。

首先，普通用户端主要面向微信小程序访问用户，其核心需求是便捷、清晰地浏览公开内容。用户进入小程序后，应能够在首页看到站点介绍、置顶文章和最新课程笔记；能够在归档页按照时间顺序查看全部公开文章；能够点击文章进入详情页，阅读文章标题、发布时间、标签、阅读量、封面图和正文内容；能够在友链页查看 GitHub、uni-app、Vue 3、Vue Router、Pinia、TypeScript、Vite、Bun、Elysia、Drizzle ORM、SQLite、Tailwind

CSS 等与项目技术栈相关的官方资料入口。小程序端页面需要适配移动端屏幕，保证文字清晰、布局美观、交互流畅。

其次，管理员端主要面向系统维护人员，其核心需求是对系统内容进行管理。管理员应能够通过后台登录系统，并在登录后进入后台管理界面。后台应提供文章管理功能，包括创建文章、编辑文章标题、正文、摘要、标签、封面、发布时间、发布状态等内容；应提供友链管理功能，支持查看、审核、修改和删除友链信息；应提供系统配置功能，支持维护站点标题、站点图标、首页横幅、管理员头像、个人简介、社交链接和页脚链接等内容；应提供邮件日志功能，用于查看系统发送邮件的记录，便于管理员了解邮件发送情况和排查问题。

再次，后端服务需要为前台小程序和后台管理端提供统一的数据接口。后端应支持公开文章列表查询、文章详情查询、归档数据查询、公开友链查询等接口，供小程序端调用；同时应支持管理员登录、文章增删改查、友链审批、配置读取与保存、文件上传、邮件日志查询等接口，供后台管理端调用。后端还需要负责数据校验、权限控制、数据库读写、图片资源保存和错误响应处理，保证系统数据的一致性和接口调用的可靠性。

最后，系统还需要满足一定的非功能性需求。性能方面，系统应能够快速加载文章列表、文章详情和友链数据，避免页面长时间停留在加载状态。安全方面，后台管理功能应要求管理员登录后才能访问，普通用户只能访问公开内容，不能直接修改系统数据。可维护性方面，系统应采用清晰的项目结构和模块化设计，将前台、小程序端、后台管理端、后端接口和数据库逻辑分开组织，便于后期维护。可用性方面，系统需要提供必要的加载状态、错误提示、空状态展示和返回导航，保证用户在不同网络环境下都有较好的使用体验。兼容性方面，小程序端应能够在微信开发者工具和真机环境中正常访问后端接口，并适配移动端阅读场景。

系统主要功能需求如下表所示：

用户角色	功能模块	需求说明
普通用户	首页浏览	查看站点信息、置顶文章和最新发布的课程笔记
普通用户	文章详情	查看文章标题、封面、发布时间、标签、阅读量和正文内容
普通用户	归档浏览	按时间顺序浏览所有公开文章，快速定位章节笔记
普通用户	友链访问	查看项目技术栈相关的官方资料入口
管理员	登录认证	通过账号和密码登录后台管理系统
管理员	文章管理	新增、编辑、发布、归档、删除文章，维护文章封面和标签
管理员	友链管理	查看、审核、修改和删除友链信息
管理员	系统配置	维护站点标题、图标、头像、横幅、社交链接和页脚信息
管理员	邮件日志	查看系统邮件发送记录和邮件状态
后端系统	接口服务	为前台和后台提供文章、友链、配置、上传和认证接口
后端系统	数据存储	使用数据库保存用户、文章、友链、配置和邮件日志等数据

综上所述，本系统需要实现一个集内容展示、内容管理、数据接口和资源维护于一体的博客小程序平台。系统既要满足普通用户在移动端阅读 Vue 3 课程笔记的需求，也要满足管理员对内容和站点信息进行维护的需求。同时，系统还需要在页面体验、数据安全、接口稳定和后期扩展方面具备一定的可靠性。

3. 概要设计

3.1 系统框架结构

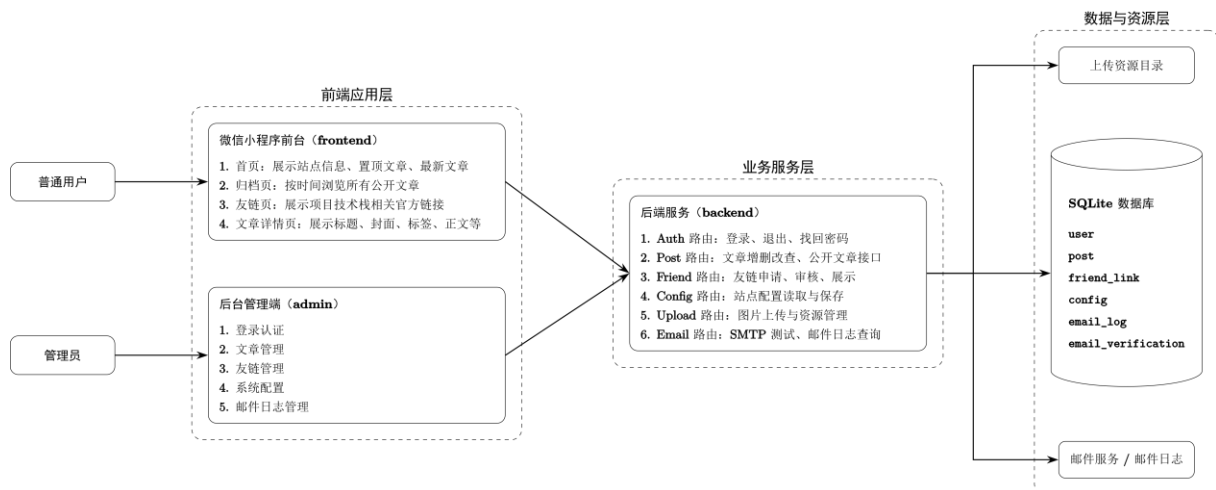
本系统采用前后端分离的总体架构，整体由微信小程序前台、后台管理端、后端服务和数据库四个部分组成。前台小程序主要面向普通访问用户，负责文章浏览、归档查看、友链访问和文章详情展示；后台管理端主要面向管理员，负责文章、友链、站点配置、邮件日志等内容的维护；后端服务负责统一提供接口、执行业务逻辑、完成权限校验并访问数据库；数据库负责持久化保存用户、文章、友链、配置和邮件日志等数据。各部分职责划分清晰，通过接口进行数据交互，既保证了系统结构的独立性，也便于后续维护和功能扩展。

系统小程序端使用 uni-app 和 Vue 3 实现。页面结构包括首页、归档页、友链页和文章详情页，其中首页、归档页、友链页配置为底部导航栏页面，文章详情页作为普通页面打开，用于展示完整文章内容。小程序端通过封装的请求模块访问后端接口，获取文章列表、归档数据、文章详情、友链列表和站点配置等信息。项目的小程序页面路径和底部 tab 配置集中在 frontend/src/pages.json 中，便于统一维护页面入口和导航结构。

后台管理端使用 Vue 3、Vite、Pinia 和 Vue Router 等技术实现。管理员登录后可以进入后台界面，对文章内容、文章封面、友链信息、系统配置、个性化信息、邮件日志等进行管理。后台管理端与小程序前台共用同一套后端服务和数据库，因此管理员在后台修改数据后，前台小程序能够通过接口读取到更新后的公开内容，保证前后台数据保持一致。

后端服务使用 Bun 作为运行时，使用 Elysia 框架提供 REST API 接口，并使用 Drizzle ORM 操作 SQLite 数据库。后端按照路由、DTO、服务、DAO、数据库表结构等层次组织代码。路由层负责接收请求并进行参数校验；服务层负责处理业务逻辑；DAO 层负责数据库读写；数据库表结构由 Drizzle 定义并通过迁移文件维护。后端还提供文件上传能力，用于保存站点图标、头像、首页横幅、文章封面和文章正文插图等资源。

系统总体框架如下：

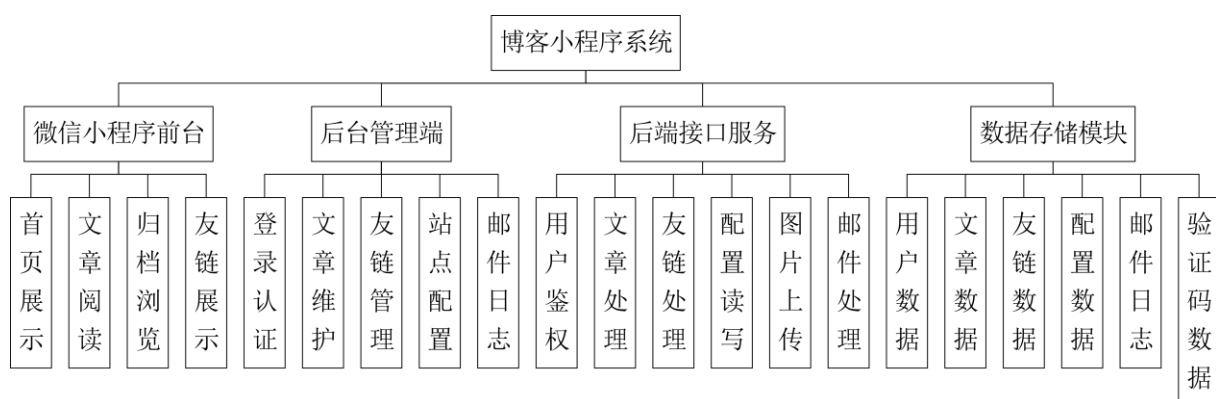


从数据流角度看，普通用户通过小程序访问公开页面，小程序向后端公开接口发起请求，后端从数据库读取已发布文章、友链和公开配置后返回给小程序展示。管理员通过后台管理端登录后，调用需要权限验证的管理接口，对文章、配置、友链等数据进行维护，后端完成数据校验后写入数据库。系统采用统一后端接口和统一数据库，可以避免多端数据不一致的问题。

3.2 系统功能结构图

根据系统使用对象和功能职责，可以将系统划分为前台展示模块、后台管理模块、后端接口模块和数据存储模块。前台展示模块面向普通用户，主要提供内容浏览能力；后台管理模块面向管理员，主要提供内容维护能力；后端接口模块是连接前端页面和数据库的核心；数据存储模块负责保存系统运行所需的各类数据，为系统稳定运行提供基础支持。

系统功能结构如下：

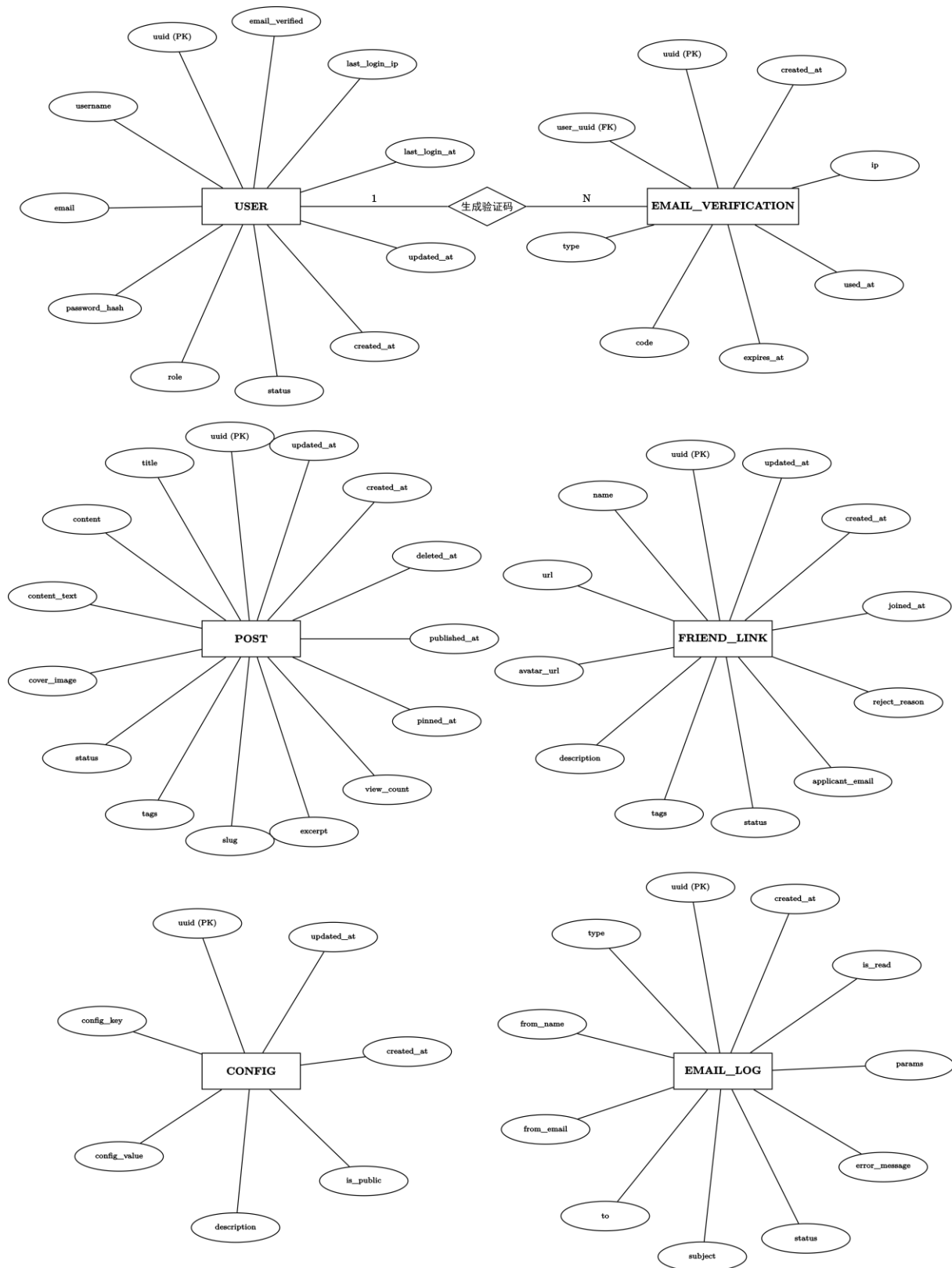


该功能结构体现了系统的主要模块关系。前台小程序侧重于内容展示和移动端阅读体验；后台管理端侧重于内容维护和系统配置；后端接口服务为前台和后台提供统一的数据访问入口；数据库保存系统运行所需的核心数据。各模块之间职责清晰，便于开发、调试和后期维护，也有利于后续功能扩展和系统优化。

3.3 系统 E-R 图

本系统数据库包括用户、文章、友链、配置、邮件日志和邮箱验证码，部分内容以 JSON 保存。其中邮箱验证码通过 user_uuid 关联用户，其余实体独立保存业务数据。

系统 E-R 关系如下：



从 E-R 图可以看出，系统核心数据表之间的关系相对清晰。文章表与友链表主要为公开展示服务，配置表为站点运行提供基础信息，用户表为后台管理提供身份基础，邮件日志表记录邮件发送过程，邮箱验证码表用于支持找回密码等安全功能。由于文章标签、友链标签和系统配置项具有一定灵活性，系统使用 JSON 字段保存这类数据，避免为每一种配置都额外建立独立表。

3.4 数据库设计

本系统使用 SQLite 作为数据库，使用 Drizzle ORM 定义数据库表结构。SQLite 适合中小型项目和本地部署场景，具有轻量、易维护、部署简单等特点。系统数据库结构由多个表组成，主要包括 user、post、friend_link、config、email_log 和 email_verification。项目的数据库入口文件统一导出这些表结构，便于后端 DAO 和服务层进行调用。

3.4.1 用户表 user

用户表用于保存后台管理员和普通用户账号信息，是系统权限认证的基础。系统登录、找回密码、邮箱验证和后台权限控制都依赖用户表中的数据。用户表中 username 和 email 均设置唯一约束，避免重复注册或重复创建管理员账号。用户密码不保存明文，而是保存加密后的密码哈希。用户表字段包括 uuid、username、email、password_hash、role、status、created_at、updated_at、last_login_at、last_login_ip、email_verified 等。

字段名	类型	说明
uuid	text	用户唯一编号，主键
username	text	用户名，唯一
email	text	邮箱地址，唯一
password_hash	text	加密后的密码
role	text	用户角色，如 admin、user
status	text	用户状态，如 active、inactive
created_at	integer	创建时间
updated_at	integer	更新时间
last_login_at	integer	最后登录时间
last_login_ip	text	最后登录 IP
email_verified	integer	邮箱是否已验证

3.4.2 文章表 post

文章表是系统最核心的数据表，用于保存 Vue 3 课程笔记文章。文章表既保存富文本内容，也保存便于搜索和统计的纯文本内容。字段 content 用于保存 ProseMirror/Tiptap 形式的富文本 JSON；字段 content_text 保存正文纯文本，便于统计

字数和搜索; 字段 `cover_image` 保存文章封面路径; 字段 `tags` 保存文章标签; 字段 `slug` 用于前台文章详情页访问。文章表还保存文章状态、阅读量、置顶时间、发布时间、删除时间等信息, 用于实现发布、归档、回收站和置顶文章等功能。你的种子文章写入逻辑中也包含 `uuid`、`title`、`content`、`content_text`、`cover_image`、`status`、`tags`、`slug`、`excerpt`、`view_count`、`pinned_at`、`published_at`、`deleted_at`、`created_at`、`updated_at` 等字段。

字段名	类型	说明
<code>uuid</code>	<code>text</code>	文章唯一编号, 主键
<code>title</code>	<code>text</code>	文章标题
<code>content</code>	<code>text/json</code>	富文本正文内容
<code>content_text</code>	<code>text</code>	正文纯文本内容
<code>cover_image</code>	<code>text</code>	文章封面图片路径
<code>status</code>	<code>text</code>	文章状态, 如 <code>draft</code> 、 <code>published</code> 、 <code>archived</code> 、 <code>deleted</code>
<code>tags</code>	<code>text/json</code>	文章标签
<code>slug</code>	<code>text</code>	文章访问路径标识
<code>excerpt</code>	<code>text</code>	文章摘要
<code>view_count</code>	<code>integer</code>	阅读量
<code>pinned_at</code>	<code>integer</code>	置顶时间
<code>published_at</code>	<code>integer</code>	发布时间
<code>deleted_at</code>	<code>integer</code>	删除时间
<code>created_at</code>	<code>integer</code>	创建时间
<code>updated_at</code>	<code>integer</code>	更新时间

3.4.3 友链表 `friend_link`

友链表用于保存友链或技术资料入口。本项目中友链页不再使用虚构站点, 而是展示 GitHub、uni-app、Vue 3、Vue Router、Pinia、TypeScript、Vite、Bun、Elysia、Drizzle ORM、SQLite、Tailwind CSS 等项目技术栈相关的官方链接。友链表保存链接名称、链接地址、头像、描述、标签、审核状态、申请邮箱、拒绝原因和加入时间等信息, 可以同时满足公开展示和后台审核管理的需求。

字段名	类型	说明
<code>uuid</code>	<code>text</code>	友链唯一编号, 主键
<code>name</code>	<code>text</code>	友链名称
<code>url</code>	<code>text</code>	友链地址
<code>avatar_url</code>	<code>text</code>	友链头像或图标地址
<code>description</code>	<code>text</code>	友链描述
<code>tags</code>	<code>text/json</code>	友链标签
<code>status</code>	<code>text</code>	审核状态, 如 <code>pending</code> 、 <code>approved</code> 、 <code>rejected</code>
<code>applicant_email</code>	<code>text</code>	申请人邮箱

reject_reason	text	拒绝原因
joined_at	integer	加入时间
created_at	integer	创建时间
updated_at	integer	更新时间

配置表用于保存系统配置项，包括外观配置、站点基础信息、个人信息和 SMTP 邮件配置等。该表采用键值形式设计，`config_key` 表示配置键名，`config_value` 使用 JSON 保存配置内容。这样可以较灵活地保存不同类型的配置数据，例如站点标题、站点图标、首页横幅、个人头像、社交链接、页脚链接、主题色和邮件服务器配置等。配置表字段包括 `uuid`、`config_key`、`config_value`、`description`、`is_public`、`created_at`、`updated_at`。其中 `config_key` 设置唯一约束，保证同一类配置只保存一条记录。

3.4.4 配置表 config

配置表用于保存系统配置项，包括外观配置、站点基础信息、个人信息和 SMTP 邮件配置等。该表采用键值形式设计，`config_key` 表示配置键名，`config_value` 使用 JSON 保存配置内容。这样可以较灵活地保存不同类型的配置数据，例如站点标题、站点图标、首页横幅、个人头像、社交链接、页脚链接、主题色和邮件服务器配置等。配置表字段包括 `uuid`、`config_key`、`config_value`、`description`、`is_public`、`created_at`、`updated_at`。其中 `config_key` 设置唯一约束，保证同一类配置只保存一条记录。

字段名	类型	说明
uuid	text	配置唯一编号，主键
config_key	text	配置键名，唯一
config_value	text/json	配置值
description	text	配置说明
is_public	integer	是否允许前台读取
created_at	integer	创建时间
updated_at	integer	更新时间

3.4.5 邮件日志表 email_log

邮件日志表用于保存系统发送邮件的记录，主要服务于后台邮件日志管理功能。系统在进行 SMTP 测试、登录提醒、找回密码、友链申请通知等操作时，可以将邮件发送结果写入该表。字段包括邮件类型、发件人名称、发件人邮箱、收件人、邮件主题、发送状态、错误信息、模板参数、是否已读和创建时间等。其中 `params` 字段使用 JSON 保存模板关键参数，便于后台重新预览邮件内容。

字段名	类型	说明
uuid	text	邮件日志唯一编号，主键
type	text	邮件类型

from_name	text	发件人名称
from_email	text	发件人邮箱
to	text	收件人邮箱
subject	text	邮件主题
status	text	发送状态, success 或 failed
error_message	text	失败原因
params	text/json	邮件模板参数快照
is_read	integer	管理员是否已读
created_at	integer	发送时间

3.4.6 邮箱验证码表 email_verification

邮箱验证码表用于保存找回密码等安全验证场景中的验证码信息。该表通过 user_uuid 字段记录验证码所属用户, 通过 type 区分验证码用途, 通过 expires_at 和 used_at 控制验证码有效期和使用状态。验证码表能够避免验证码重复使用, 并为密码重置流程提供数据支持。

字段名	类型	说明
uuid	text	验证码记录唯一编号, 主键
user_uuid	text	关联用户编号
type	text	验证码用途
code	text	6 位数字验证码
expires_at	integer	过期时间
used_at	integer	使用时间
ip	text	请求 IP 地址
created_at	integer	创建时间

4. 系统详细设计

4.1 小程序首页模块设计

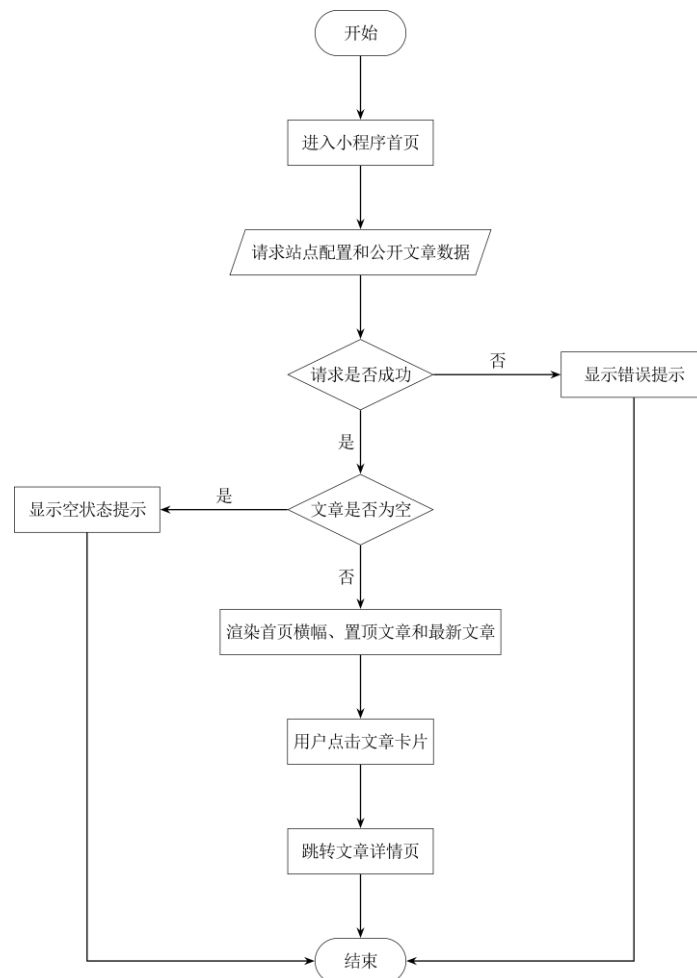
小程序首页模块是普通用户进入系统后的主要入口, 主要负责展示博客系统的基本信息、首页横幅、置顶文章和最新文章列表。用户打开微信小程序后, 系统会自动向后端请求公开文章数据和站点配置数据, 并将获取到的数据展示在首页界面中。

首页模块主要包括站点信息展示、文章列表展示、文章卡片跳转和加载状态处理等功能。系统在加载过程中会显示加载提示; 如果接口请求失败, 则显示错误提示; 如果数据加载成功, 则按照页面结构展示站点标题、简介、文章封面、文章标题、摘要、发布时间、阅读量等信息。用户点击某一篇文章后, 系统会根据文章的 slug 跳转到文章详情页面。

该模块的设计重点是保证首页内容加载清晰、文章展示美观、页面跳转准确，并且在网络异常或数据为空时能够给出合理的界面反馈。

首页模块的基本流程为：用户进入小程序首页后，系统初始化页面数据，并调用后端公开文章接口获取文章列表。如果请求成功，则判断文章数据是否为空；若不为空，则渲染置顶文章和最新文章列表；若为空，则显示空状态提示。如果请求失败，则显示错误信息。用户点击文章卡片后，页面跳转至对应文章详情页。

流程图如下：



4.2 文章详情模块设计

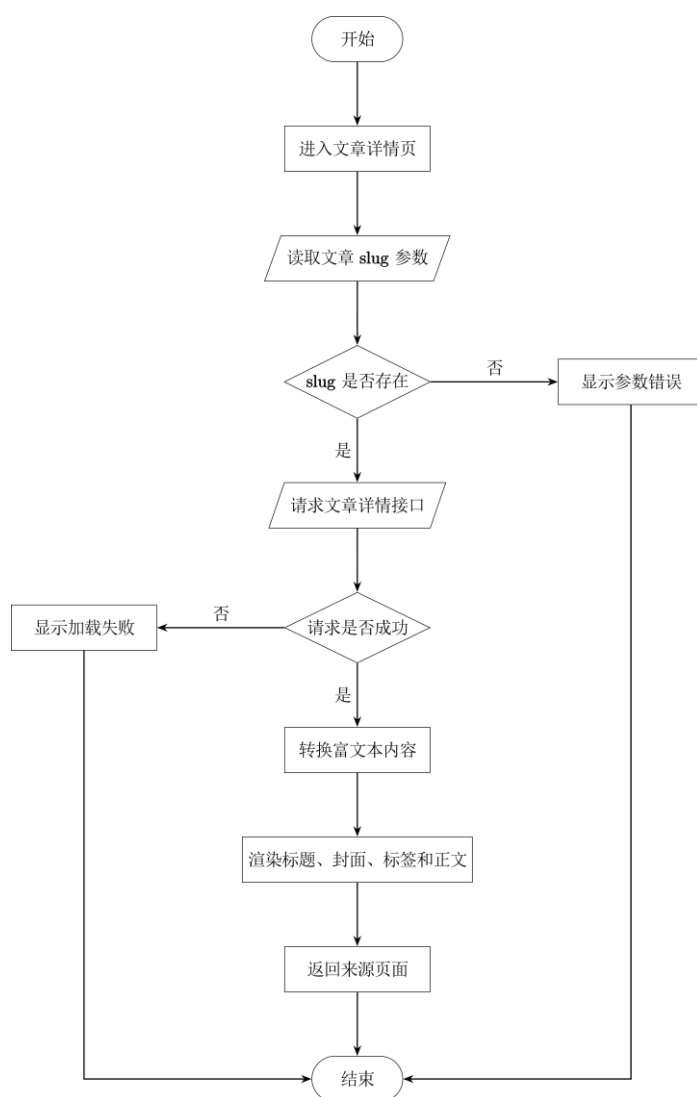
文章详情模块用于展示某一篇文章的完整内容，是小程序前台的核心阅读页面。用户从首页或归档页点击文章后，系统会携带文章的 slug 参数进入详情页面。详情页面根据该参数向后端请求对应文章的详细数据。

文章详情模块主要展示文章标题、封面图、发布时间、字数、阅读量、标签和正文内容。正文内容由后端返回的富文本数据转换为小程序可展示的 rich-text 节点后进行渲染。该模块还需要处理加载中、加载失败、文章不存在等情况。

为了提升用户体验，文章详情页还设计了返回来源页面的功能。如果用户从首页或归档页进入文章详情，点击返回按钮时可以回到原来的页面；如果没有明确来源，则默认返回上一页或首页。

文章详情模块的流程为：用户点击文章后进入详情页，系统读取页面参数中的 slug，判断参数是否存在。如果参数不存在，则显示错误提示；如果参数存在，则调用后端接口获取文章详情。请求成功后，系统将文章正文转换为富文本节点并渲染页面；请求失败时显示错误提示。用户阅读完成后，可以通过返回按钮回到来源页面。

流程图如下：



4.3 归档模块设计

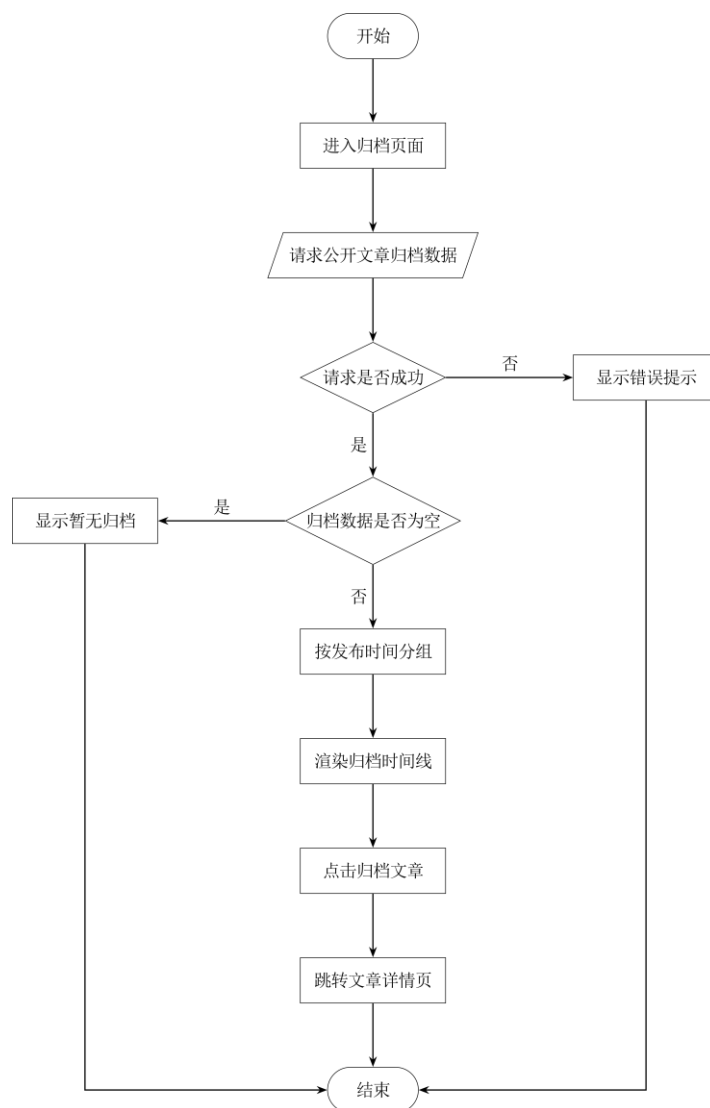
归档模块用于按照时间顺序展示系统中的公开文章，方便用户按发布时间浏览课程笔记和博客内容。该模块从后端获取所有公开文章数据，并根据文章发布时间进行分组展示。

归档页面相比首页更加注重文章的时间组织方式，通常会按照年份、月份或时间线形式展示文章标题。用户可以通过归档列表快速查看不同时间发布的文章，并点击某一条归档记录进入文章详情页。

该模块的设计重点是保证文章时间分组清晰、列表加载稳定、跳转路径正确。当后端没有公开文章数据时，页面需要显示暂无归档内容的提示。

用户进入归档页面后，系统调用后端公开文章归档接口。接口返回文章数据后，前端根据发布时间对文章进行分组处理，并渲染归档时间线。用户点击归档条目后，系统根据文章 slug 跳转到文章详情页。如果接口请求失败，则显示错误提示。

流程图如下：



4.4 后台登录认证模块设计

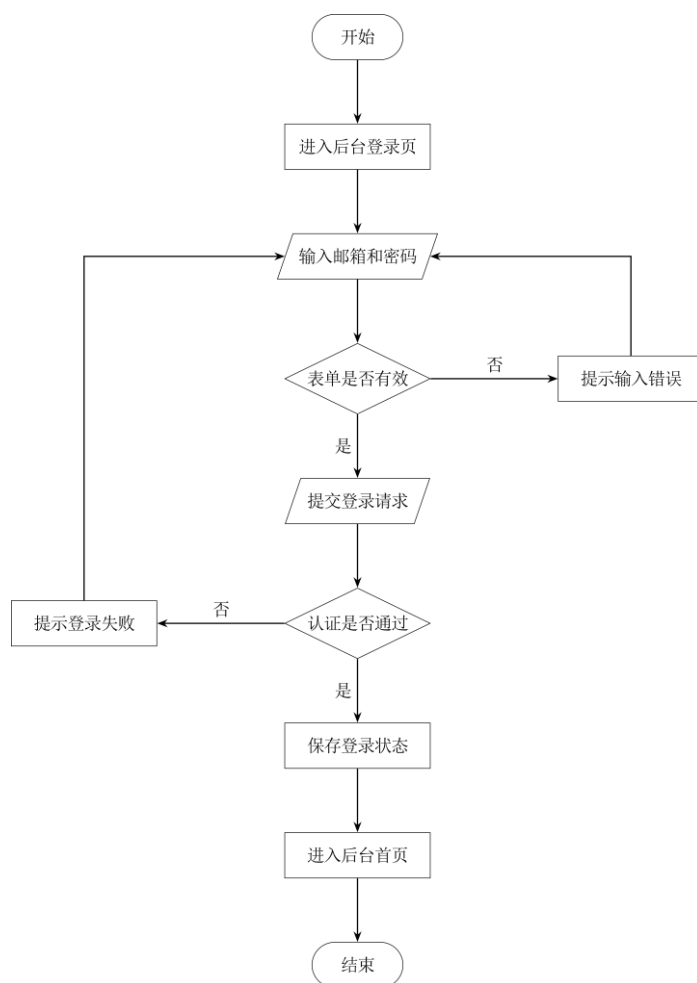
后台登录认证模块用于保证管理端系统的安全性，只有管理员用户通过身份验证后才能进入后台管理界面。用户在登录页面输入邮箱和密码后，系统会将登录信息提交到后端认证接口，由后端校验用户身份。

如果登录成功，后端返回认证结果，前端保存登录状态并跳转到后台首页；如果登录失败，页面显示错误提示。后台页面在访问时还会检查当前用户信息，如果用户未登录或登录状态失效，则自动跳转到登录页面。

该模块还包含退出登录和忘记密码功能。管理员可以主动退出后台系统；如果忘记密码，可以通过邮箱验证码进行密码重置。

管理员进入后台登录页后，输入账号和密码并提交表单。前端先进行基本表单校验，判断邮箱和密码是否为空。校验通过后，请求后端登录接口。后端验证成功则返回用户信息，前端保存状态并进入后台首页；验证失败则显示登录失败提示。

流程图如下：



4.5 后台文章管理模块设计

文章管理模块是后台管理端的核心模块，主要用于管理员对博客文章进行创建、编辑、发布、归档和删除等操作。管理员可以在后台查看文章列表，也可以进入文章编辑页面修改文章标题、摘要、正文内容、标签、封面图和发布状态。

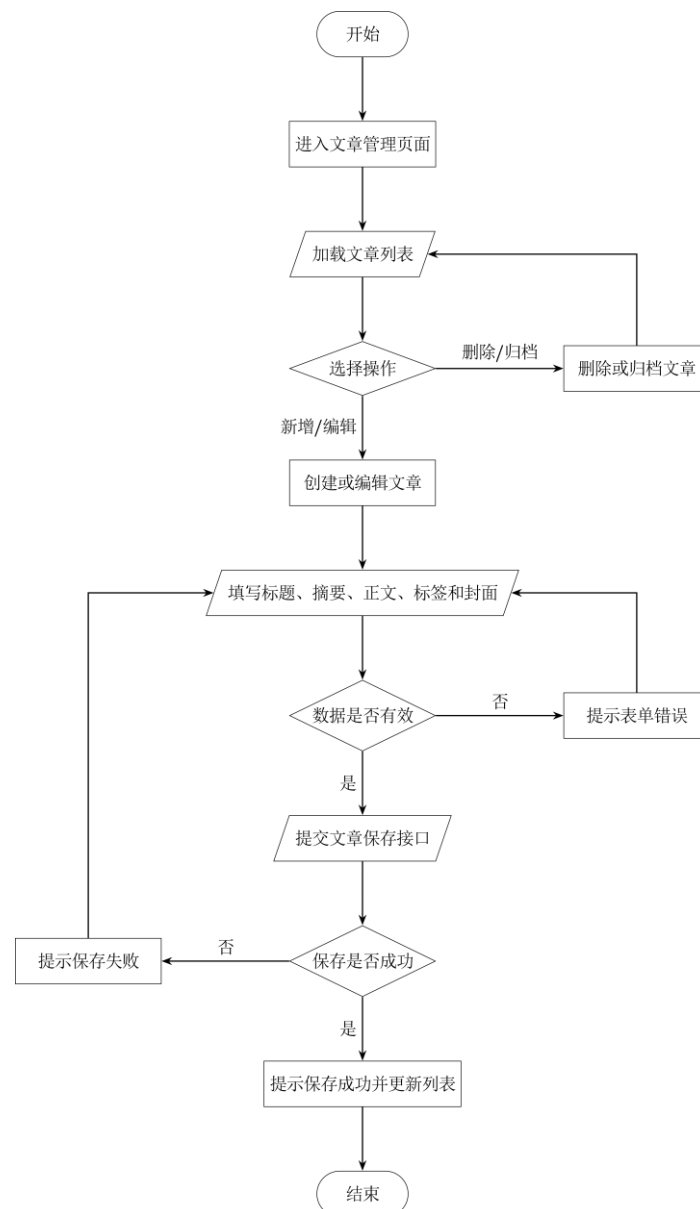
文章正文采用富文本编辑方式，便于管理员编辑课程笔记、插入图片和代码内容。文章封面和正文图片通过上传接口保存到后端上传资源目录中，数据库中只保存图片访

问路径。文章保存时，系统会将文章内容、状态、标签、封面等信息提交给后端，由后端完成数据校验和数据库写入。

该模块的设计重点是保证文章编辑过程稳定、数据保存准确、图片上传路径正确，并且能够区分草稿、发布、归档、删除等不同状态。

管理员进入文章管理页面后，系统加载文章列表。管理员可以选择创建新文章或编辑已有文章。进入编辑页面后，填写文章标题、摘要、正文、标签、封面等内容，并选择保存或发布。系统先进行表单校验，校验通过后调用后端文章保存接口。如果保存成功，则返回文章列表或提示保存成功；如果保存失败，则显示错误提示。

流程图如下：



4.6 友链管理与展示模块设计

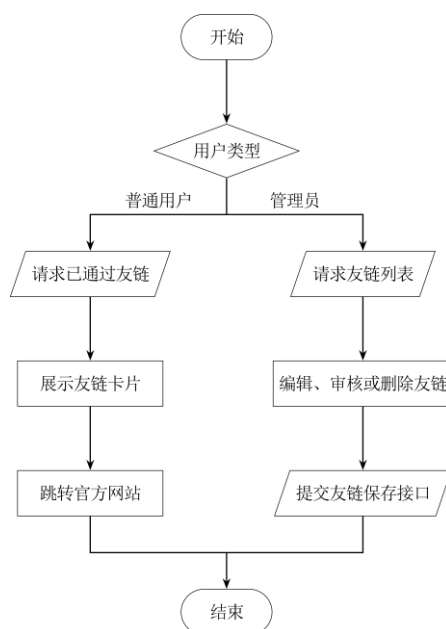
友链模块分为小程序前台展示和后台管理两部分。小程序前台主要负责展示已经审核通过的友链信息，包括链接名称、头像、描述、标签和官方网站地址。用户可以通过友链页面查看项目中使用到的技术栈相关官方链接，并跳转到对应网站。

后台管理端主要负责友链数据维护。管理员可以查看友链列表，修改友链名称、链接地址、头像、描述、标签和审核状态，也可以删除无效友链。后端负责提供友链查询、保存、审核和删除接口，并将友链数据保存到数据库中。

该模块的设计重点是保证前台只展示审核通过的友链，后台可以对友链信息进行统一管理，从而提高系统内容维护的灵活性。

前台用户进入友链页面后，系统请求已通过友链数据并展示；如果数据为空，则显示暂无友链提示。管理员进入后台友链管理页面后，可以查看全部友链数据，并对友链进行编辑、审核或删除操作。保存成功后，前台展示内容会随数据库数据同步更新。

流程图如下：



4.7 系统配置模块设计

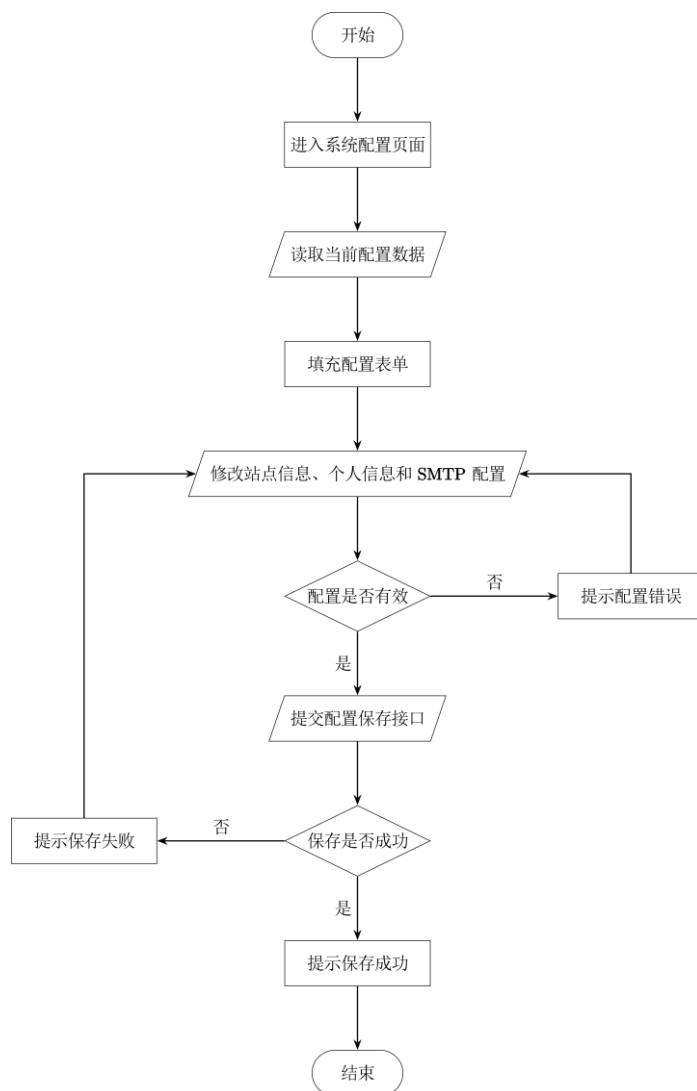
系统配置模块用于管理博客小程序中的站点基础信息和个性化展示内容。管理员可以在后台配置站点标题、站点图标、首页横幅、个人头像、个人简介、社交链接、页脚链接以及 SMTP 邮件服务参数等内容。

系统配置数据统一存储在数据库的配置表中，前台小程序在页面加载时通过配置接口读取这些信息，并根据配置内容动态展示页面。这样可以避免将站点信息写死在前端代码中，提高系统维护的灵活性。

该模块的设计重点是实现配置数据的统一读取与保存，使管理员可以通过后台界面直接修改站点展示内容，而无需重新修改代码或重新发布前端程序。

管理员进入系统配置页面后,系统从后端读取当前配置数据,并填充到配置表单中。管理员修改配置内容后点击保存,系统对表单数据进行校验,并提交到后端配置保存接口。保存成功后,数据库中的配置内容被更新,小程序前台再次请求配置接口时即可展示最新信息。

流程图如下:



4.8 图片上传模块设计

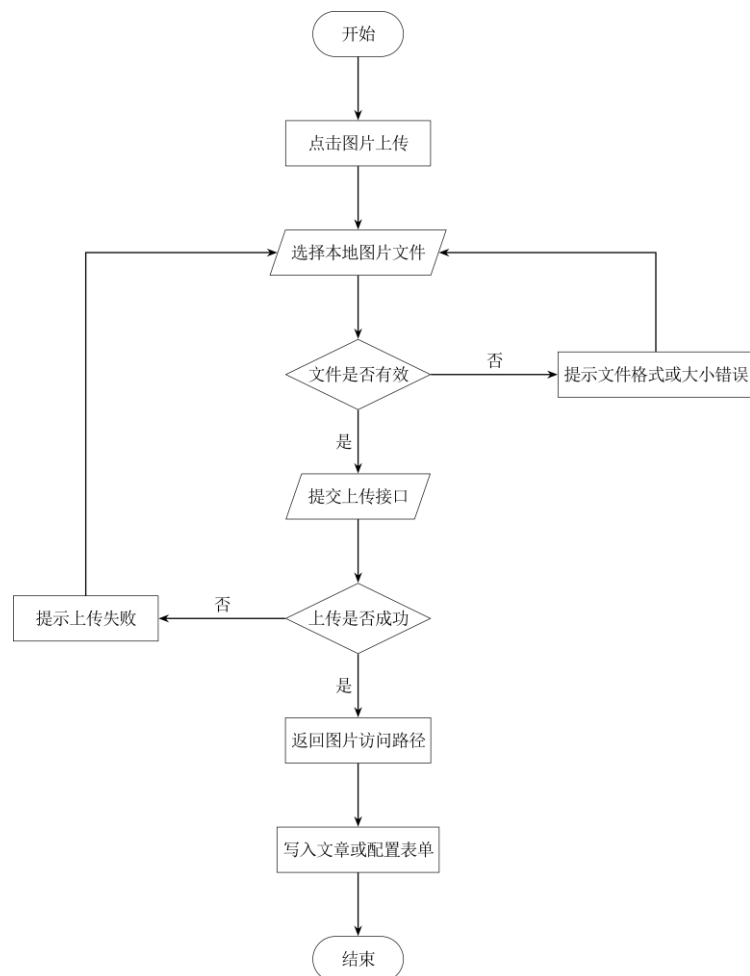
图片上传模块主要用于后台管理端上传文章封面、正文图片、站点图标、首页横幅、个人头像和社交图标等资源。管理员选择本地图片后,前端会将图片文件提交给后端上传接口,后端将文件保存到上传资源目录中,并返回图片的访问路径。

数据库中不直接保存图片文件本身,而是保存图片路径。前台或后台页面在展示图片时,根据数据库中的路径访问后端静态资源接口,从而显示对应图片。

该模块的设计重点是保证上传文件格式正确、保存路径统一、返回地址可访问，并且避免前端直接依赖本地文件路径。通过该模块，系统可以实现文章内容和站点展示资源的动态维护。

管理员点击上传按钮后，系统打开文件选择窗口。用户选择图片后，前端检查文件格式和大小，校验通过后将图片提交给后端上传接口。后端保存文件并返回图片路径，前端将该路径写入表单数据中。管理员保存文章或配置后，图片路径被写入数据库，前台页面即可正常访问图片。

流程图如下：



5. 系统实现

5.1 小程序文章详情阅读功能

文章详情页是小程序端的核心功能之一。用户在首页或归档页点击文章后，系统会根据文章的 slug 跳转到详情页。详情页需要完成文章数据请求、加载状态展示、错误提示、文章标题显示、发布时间显示、阅读量显示、标签显示、封面图显示和富文本正文渲染等功能。

该功能的实现思路如下：首先通过 `onLoad` 生命周期接收页面参数中的 `slug`，然后调用 `getPostBySlug` 请求后端公开文章详情接口；请求过程中使用 `loading` 控制加载状态，发生异常时通过 `error` 展示错误信息；请求成功后将文章数据保存到 `post` 响应式变量中。页面渲染时通过计算属性生成封面图地址、格式化日期、字数、阅读量和富文本节点，最后在模板中展示完整文章内容。

关键代码如下：

```
<script setup lang="ts">
// 引入 Vue 的 computed 和 ref，用于创建响应式数据和计算属性
import { computed, ref } from "vue";

// 引入 uni-app 页面生命周期和分享钩子
import { onLoad, onShareAppMessage } from "@dcloudio/uni-app";

// 引入页面头部组件，用于显示标题、副标题和返回按钮
import MiniHeader from "@components/MiniHeader.vue";

// 引入状态组件，用于显示加载中和错误提示
import StateBlock from "@components/StateBlock.vue";

// 引入根据 slug 获取文章详情的接口方法
import { getPostBySlug } from "@services/posts";

// 引入文章详情类型，保证 TypeScript 类型安全
import type { PublicPostDetail } from "@types/blog";

// 引入格式化工具：数字压缩、日期格式化、资源地址处理
import { compactNumber, formatDate, resolveAssetUrl } from "@utils/format";

// 引入来源 tab 地址工具，保证返回时可以回到正确页面
import { getSourceTabUrl } from "@utils/navigation";

// 引入富文本转换工具，将后端保存的 ProseMirror JSON 转换为 rich-text 可渲染节点
import { toRichTextNodes } from "@utils/content";

// 保存当前文章的 slug，slug 是文章详情接口的唯一访问标识
const slug = ref("");

// 保存来源页面，可能来自首页 home，也可能来自归档 archive
```

```

const origin = ref("home");

// 保存后端返回的文章详情数据，初始值为空
const post = ref<PublicPostDetail | null>(null);

// loading 用于控制“正在打开文章”的加载状态
const loading = ref(false);

// error 用于保存请求失败时的错误提示
const error = ref("");

// 计算文章封面图地址，如果后端返回的是相对路径，则通过 resolveAssetUrl 转成完整地址
const cover = computed(() => resolveAssetUrl(post.value?.coverImage));

// 计算文章发布时间，将时间戳或日期字符串格式化为页面展示用的日期
const date = computed(() => formatDate(post.value?.publishedAt));

// 计算文章字数，如果后端没有返回，则默认显示 0
const words = computed(() => compactNumber(post.value?.wordCount ?? 0));

// 计算文章阅读量，如果后端没有返回，则默认显示 0
const views = computed(() => compactNumber(post.value?.viewCount ?? 0));

// 将文章正文内容转换为小程序 rich-text 组件可以渲染的节点
const richNodes = computed(() => toRichTextNodes(post.value?.content));

// 取文章第一个标签作为封面角标，如果没有标签则使用默认 Essay
const primaryTag = computed(() => post.value?.tags?.[0] || "Essay");

// 定义文章详情请求方法
async function fetchPost() {
  // 如果页面参数中没有 slug，说明文章地址不完整，直接显示错误信息
  if (!slug.value) {
    error.value = "文章地址不完整";
    return;
  }

  // 请求开始前打开 loading，并清空历史错误

```



```

loading.value = true;
error.value = "";

try {
  // 调用文章详情接口，根据 slug 获取文章完整数据
  const data = await getPostBySlug(slug.value);

  // 将返回的文章对象保存到响应式变量中，页面会自动更新
  post.value = data.post;
} catch (err) {
  // 如果接口请求失败，提取错误信息并显示到页面
  error.value = err instanceof Error ? err.message : "文章加载失败";
} finally {
  // 不论请求成功还是失败，最后都关闭 loading 状态
  loading.value = false;
}
}

// 返回按钮逻辑
function back() {
  // 获取当前小程序页面栈
  const pages = getCurrentPages();

  // 如果页面栈长度大于 1，说明是从其他页面 navigateTo 进入，可以直接返回上一页
  if (pages.length > 1) {
    uni.navigateBack();
    return;
  }

  // 如果页面栈长度不足，说明可能是分享进入或直接打开，此时切换到来源 tab 页面
  uni.switchTab({ url: getSourceTabUrl(origin.value) });
}

// 页面加载时读取路由参数
onLoad((query) => {
  // 读取并解码文章 slug
  slug.value = decodeURIComponent(String(query?.slug ?? ""));
}

```

```

// 读取来源页面，默认来源为 home
origin.value = String(query?.from ?? "home");

// 调用文章详情请求方法
void fetchPost();
});

// 配置微信小程序分享内容
onShareAppMessage(() => ({
  // 分享标题优先使用文章标题，没有文章时使用默认标题
  title: post.value?.title || "分享文章",

  // 分享路径中携带 slug，方便其他用户打开同一篇文章
  path: `/pages/post/detail?slug=${encodeURIComponent(slug.value)}`,
}));
</script>

```

页面模板部分如下：

```

<template>
  <!-- dream-page 是全局页面背景类，page 用于控制页面底部安全距离 -->
  <view class="dream-page page">
    <!-- 顶部页面标题区域，show-back 表示显示返回按钮 -->
    <MiniHeader
      eyebrow="Reading"
      title="阅读文章"
      subtitle="完整阅读当前文章。"
      show-back
      compact
      @back="back"
    />

    <!-- 文章请求中显示加载状态 -->
    <StateBlock v-if="loading" title="正在打开文章" loading />

    <!-- 请求失败时显示错误信息 -->
    <StateBlock v-else-if="error" title="打开失败" :description="error" />

    <!-- 请求成功后显示文章主体 -->
    <view v-else-if="post" class="article porcelain-card">

```

```

<!-- 文章头部信息，包括日期、标题、字数、阅读量和标签 -->
<view class="article-head">
  <!-- 发布时间 -->
  <view class="article-chip">
    <text>{{ date }}</text>
  </view>

  <!-- 文章标题 -->
  <text class="title">{{ post.title || "未命名文章" }}</text>

  <!-- 文章元信息 -->
  <view class="meta">
    <text>{{ words }} 字</text>
    <text class="dot" />
    <text>{{ views }} 阅读</text>
  </view>

  <!-- 文章标签列表 -->
  <view v-if="post.tags?.length" class="tags">
    <text v-for="tag in post.tags" :key="tag" class="tag">
      {{ tag }}
    </text>
  </view>
</view>

<!-- 文章正文区域 -->
<view class="article-body">
  <!-- 如果文章有封面图，则把封面图显示在正文开头 -->
  <view v-if="cover" class="content-cover-panel">
    <image :src="cover" mode="aspectFill" class="cover" />
    <view class="cover-mask" />
    <text class="cover-tag">{{ primaryTag }}</text>
  </view>

  <!-- 渲染后端保存的富文本正文 -->
  <rich-text :nodes="richNodes" />
</view>
</view>

```

```
</view>
</template>
```

该功能实现后，用户可以从首页、归档页进入文章详情页，并完整阅读文章内容。页面中同时处理了加载中、请求失败和请求成功三种状态，避免页面出现空白或无响应情况。封面图放置在正文开头，使文章视觉层次更加清晰，也减少了封面图和正文图片重复显示的问题。

5.2 小程序接口请求与文章列表功能

小程序端需要频繁请求后端接口，例如首页文章列表、归档列表、文章详情、友链列表和站点配置等。为了避免在每个页面重复编写 `uni.request`，系统封装了统一请求函数。该函数负责拼接基础接口地址、处理查询参数、设置请求方法、解析后端统一响应格式，并在请求失败时返回错误信息。

该功能的主要作用是提高接口调用的复用性和稳定性。页面只需要调用对应的业务接口方法，不需要关心底层请求细节。例如文章列表调用 `getPublicPosts`，归档页调用 `getArchivePosts`，文章详情页调用 `getPostBySlug`。这样可以使页面代码更加简洁，也方便后期统一修改接口地址或错误处理逻辑。

关键代码如下：

```
// 引入后端 API 基础地址
// 该地址在本地开发时可以配置为 http://localhost:3001
// 真机调试时需要改为电脑局域网 IP，例如 http://10.162.92.32:3001
import { API_BASE_URL } from "@config";

// 定义请求方法类型，限制只能使用常见 HTTP 方法
type Method = "GET" | "POST" | "PUT" | "DELETE";

// 定义后端统一响应格式
// success 表示请求是否成功，data 保存真正的业务数据
interface ApiEnvelope<T> {
  success: boolean;
  data: T;
}

// 将 query 对象转换为 URL 查询字符串
function joinQuery(query?: Record<string, unknown>) {
  // 如果没有 query 参数，直接返回空字符串
  if (!query) return "";
```

```

// 遍历 query 对象，过滤 undefined、null 和空字符串
const params = Object.entries(query)
  .filter(
    ([, value]) => value !== undefined && value !== null && value !== ""
  )
  .map(
    ([key, value]) =>
      // 对 key 和 value 做 URL 编码，避免中文、空格等字符导致请求错误
      `${encodeURIComponent(key)}=${encodeURIComponent(String(value))}`,
  );

// 如果存在参数，则用 ? 拼接；否则返回空字符串
return params.length ? `?${params.join("&")}` : "";
}

// 封装统一请求方法
export function request<T>(
  path: string,
  options: {
    // 请求方法，默认使用 GET
    method?: Method;

    // 请求体数据，例如 POST 或 PUT 时使用
    data?: unknown;

    // 查询参数，例如分页 page、pageSize、keyword 等
    query?: Record<string, unknown>;
  } = {},
) {
  // 拼接完整请求地址
  const url = `${API_BASE_URL}${path}${joinQuery(options.query)}`;

  // uni.request 使用回调形式，因此这里包装成 Promise，方便页面中使用 async/await
  return new Promise<T>((resolve, reject) => {
    uni.request({
      // 请求地址
      url,
    });
  });
}

```

```

// 请求方法，没有传入时默认 GET
method: options.method ?? "GET",

// 请求体数据
data: options.data as
  | string
  | ArrayBuffer
  | UniApp.RequestOptions["data"],

// 设置 JSON 请求头
header: {
  "Content-Type": "application/json",
},

// 请求成功回调
success: (res) => {
  // 取出响应体
  const body = res.data as ApiEnvelope<T> | T | undefined;

  // 如果 HTTP 状态码不是 2xx，说明请求失败
  if (res.statusCode < 200 || res.statusCode >= 300) {
    // 优先读取后端返回的错误信息
    const message =
      typeof body === "object" &&
      body &&
      "data" in body &&
      typeof (body as { data?: { message?: string } }).data?.message ===
        "string"
      ? (body as { data: { message: string } }).data.message
      : "请求失败，请稍后重试";

    // 抛出错误，交给页面 catch 处理
    reject(new Error(message));
    return;
  }

  // 如果后端返回的是统一包装格式 { success, data }
  if (body && typeof body === "object" && "success" in body) {

```

```

    const envelope = body as ApiEnvelope<T>;

    // success 为 true 时返回真正的数据
    if (envelope.success) {
        resolve(envelope.data);
    } else {
        // success 为 false 时提示请求失败
        reject(new Error("请求失败，请稍后重试"));
    }
    return;
}

// 兜底处理：如果后端返回的不是包装格式，则直接返回 body
resolve(body as T);
},

// 网络层失败，例如后端未启动、地址不可访问、真机无法访问 localhost 等
fail: (error) => {
    reject(error);
},
});
});
}

```

文章接口封装如下：

```

// 引入文章相关类型
import type { ArchivePost, PostListItem, PublicPostDetail } from "@types/blog";

// 引入统一请求函数
import { request } from "../request";

// 获取公开文章列表
export function getPublicPosts(query: {
    // 当前页码
    page?: number;

    // 每页数量
    pageSize?: number;

```

```

    // 搜索关键词，可用于按标题或内容搜索
    keyword?: string;
  }) {
    // 请求后端公开文章列表接口
    return request<{ list: PostListItem[]; total: number }>("/api/public/posts", {
      query,
    });
  }

  // 获取归档文章列表
  export function getArchivePosts() {
    // 归档页使用轻量列表，不需要加载完整正文
    return request<{ list: ArchivePost[] }>("/api/public/posts/archive");
  }

  // 根据 slug 获取文章详情
  export function getPostBySlug(slug: string) {
    // slug 需要 encodeURIComponent，避免中文或特殊字符导致 URL 错误
    return request<{ post: PublicPostDetail }>(
      `/api/public/posts/${encodeURIComponent(slug)}`,
    );
  }
}

```

通过以上封装，小程序页面不需要直接操作 `uni.request`，只需要调用语义明确的业务函数即可。例如首页调用 `getPublicPosts` 获取最新文章，归档页调用 `getArchivePosts` 获取时间线数据，详情页调用 `getPostBySlug` 获取正文内容。该设计提高了代码复用性，也使接口错误处理更加统一。

5.3 后台文章封面上传功能

后台文章管理是系统的重要组成部分，其中封面图上传功能用于为文章设置封面，提高文章列表和详情页的视觉效果。管理员在后台编辑文章时，可以选择本地图片作为文章封面。系统会先将图片上传到后端，后端保存图片后返回图片访问地址，前端再将该地址保存到文章数据中，从而实现文章内容与图片资源的关联。

该功能的实现流程为：管理员选择文件后触发 `onFileChange`；前端调用 `uploadPostCover` 上传封面图；上传成功后返回图片 URL；前端将 URL 赋值给 `coverImage`；最后调用 `savePost` 保存文章封面字段。这样可以保证图片文件和数据库记录保持一致，避免文章保存后出现封面丢失或路径错误的问题，同时也便于前台页面正确加载并展示文章封面。

关键代码如下：

```
<script setup lang="ts">
// 引入图片图标，用于设置面板中显示封面上传入口
import { RiImageFill } from "@remixicon/vue";

// 引入设置项容器组件，用于统一后台设置面板样式
import PostEditorSettingItem from "../PostEditorSettingItem.vue";

// 引入通用图片上传组件
import ImageUpload from "@components/base/upload/ImageUpload.vue";

// 引入国际化工具，用于显示中文或英文提示
import { useLang } from "@composables/lang.hook";

// 引入图片上传 hook，封装上传 loading、文件选择和上传流程
import { useImageUpload } from "@composables/upload.hook";

// 引入提示工具，用于上传失败或保存失败时显示错误信息
import { useToast } from "@composables/toast.hook";

// 引入文章封面上传接口
import { uploadPostCover } from "@api/upload.api";

// 引入文章保存接口，用于把封面 URL 持久化到数据库
import { savePost } from "@api/post.api";

// 获取国际化函数
const { t } = useLang();

// 当前组件需要接收文章 uuid
// uuid 用于告诉后端当前封面属于哪一篇文章
const props = defineProps<{
  uuid: string;
}>();

// 定义 v-model，保存当前文章封面图 URL
const coverImage = defineModel<string | null>({ default: null });
```

```

// 初始化图片上传 hook
const cover = useImageUpload();

// 上传状态，用于控制按钮 loading 和防止重复上传
const coverLoading = cover.loading;

// 上传组件引用，用于触发文件选择
const coverUploadRef = cover.uploadRef;

// 当用户选择封面文件时执行
const onFileChange = (file: File) => {
  // 调用通用上传流程
  cover.handleUpload(
    // 用户选择的图片文件
    file,

    // 第二个参数是实际上传函数
    // 将文件包装成后端 DTO 需要的 cover 字段
    (f) => uploadPostCover(props.uuid, { cover: f }),

    // 第三个参数是上传成功后的回调
    (url) => {
      // 将后端返回的图片 URL 写入 v-model
      coverImage.value = url;

      // 调用文章保存接口，将封面 URL 写入数据库
      savePost(props.uuid, { coverImage: url }).catch((e: unknown) => {
        // 如果保存失败，提取错误信息
        const msg = typeof e === "string" ? e : (e as any)?.message || "Error";

        // 显示错误提示，方便管理员排查
        useToast.error(t(`api.errors.${msg}`));
      });
    },
  );
};
</script>

```

页面模板如下：

```

<template>
  <!-- PostEditorSettingItem 负责统一显示设置项标题、图标和提示 -->
  <PostEditorSettingItem
    :label="t('views.admin.PostEditor.settingsPanel.coverImage.label')"
    :tooltip="t('views.admin.PostEditor.settingsPanel.coverImage.tooltip')"
  >
    <!-- 设置项左侧图标 -->
    <template #icon>
      <RiImageFill class="w-4 h-4 text-fg-subtle" />
    </template>

    <!-- 图片上传组件 -->
    <ImageUpload
      ref="coverUploadRef"
      v-model="coverImage"
      :loading="coverLoading"
      width="w-full"
      height="h-32"
      rounded-class="rounded-xl"
      @change="onFileChange"
    >
      <!-- 未上传图片时显示的占位内容 -->
      <template #icon>
        <div class="flex flex-col items-center gap-2">
          <RiImageFill class="w-8 h-8 text-fg-subtle/30" />
          <p class="text-[10px] text-fg-subtle/50 text-center leading-tight">
            {{ t("views.admin.PostEditor.settingsPanel.coverImage.hint") }}
          </p>
        </div>
      </template>
    </ImageUpload>
  </PostEditorSettingItem>
</template>

```

该功能实现后，管理员可以在后台文章编辑界面直接上传封面图。上传成功后，系统会自动将封面图地址保存到文章记录中，小程序端在首页卡片、归档列表或文章详情页中即可读取并显示该封面图。该功能提高了文章内容的展示效果，也使后台内容维护更加方便。

5.4 文章初始化与课程笔记数据写入功能

为了让系统在首次运行时拥有可展示的数据，项目设计了文章种子数据写入功能。该功能用于将 Vue 3 课程章节内容整理成系统文章，并写入数据库。系统通过 `contentBlocks` 描述文章内容结构，再将其转换为富文本 JSON 存入数据库，同时生成纯文本内容用于搜索、摘要和字数统计。

该功能的实现思路是：先定义文章块类型，例如标题、段落、无序列表、有序列表、引用、代码块和图片；再定义不同节点的转换函数；然后通过 `buildContent` 将内容块转换为富文本结构；通过 `contentText` 提取纯文本；最后在 `upsertPost` 中执行 SQL 写入文章表。为了防止重复写入，系统使用 `ON CONFLICT(slug) DO UPDATE`，当文章 `slug` 已存在时更新文章内容。

关键代码如下：

```
// 定义文章内容块类型
// 每种 type 对应一种文章内容结构
type RichTextBlock =
  // 二级或三级标题
  | { type: "heading"; level: 2 | 3; text: string }

  // 普通段落
  | { type: "paragraph"; text: string }

  // 无序列表
  | { type: "bulletList"; items: string[] }

  // 有序列表
  | { type: "orderedList"; items: string[] }

  // 引用块
  | { type: "blockquote"; text: string }

  // 代码块
  | { type: "codeBlock"; text: string; language?: string }

  // 图片节点
  | { type: "image"; src: string; alt?: string; width?: number };

// 生成文本节点
const textNode = (text: string) => ({ type: "text", text });
```

```

// 根据文章 uuid 生成封面路径
const postCover = (uuid: string) => `/api/uploads/posts/${uuid}/cover.webp`;

// 生成段落节点
const paragraphNode = (text: string) => ({
  type: "paragraph",
  content: [textNode(text)],
});

// 生成标题节点
const headingNode = (level: 2 | 3, text: string) => ({
  type: "heading",
  attrs: { level },
  content: [textNode(text)],
});

// 生成列表节点
function listNode(type: "bulletList" | "orderedList", items: string[]) {
  return {
    type,
    content: items.map((item) => ({
      type: "listItem",
      content: [paragraphNode(item)],
    })),
  };
}

// 生成引用节点
function blockquoteNode(text: string) {
  return {
    type: "blockquote",
    content: [paragraphNode(text)],
  };
}

// 生成代码块节点
function codeBlockNode(text: string, language = "typescript") {

```

```

return {
  type: "codeBlock",
  attrs: { language },
  content: [textNode(text)],
};
}

// 生成图片节点
function imageNode(src: string, alt = "", width?: number) {
  return {
    type: "paragraph",
    content: [
      {
        type: "image",
        attrs: {
          src,
          alt,
          title: null,
          width: width ?? null,
        },
      },
    ],
  };
}

// 将自定义 contentBlocks 转换为富文本 JSON
function buildContent(blocks: RichTextBlock[]) {
  return {
    type: "doc",
    content: blocks.map((block) => {
      switch (block.type) {
        // 标题块转换为 heading 节点
        case "heading":
          return headingNode(block.level, block.text);

        // 段落块转换为 paragraph 节点
        case "paragraph":
          return paragraphNode(block.text);
      }
    });
  };
}

```

```

// 无序列表转换为 bulletList 节点
case "bulletList":
    return listNode("bulletList", block.items);

// 有序列表转换为 orderedList 节点
case "orderedList":
    return listNode("orderedList", block.items);

// 引用块转换为 blockquote 节点
case "blockquote":
    return blockquoteNode(block.text);

// 代码块转换为 codeBlock 节点
case "codeBlock":
    return codeBlockNode(block.text, block.language);

// 图片块转换为 image 节点
case "image":
    return imageNode(block.src, block.alt, block.width);
}
}),
};
}

// 提取文章纯文本
function contentText(blocks: RichTextBlock[]) {
    return blocks
        .flatMap((block) => {
            // 如果内容块中有 text 字段，则提取 text
            if ("text" in block) return [block.text];

            // 如果内容块中有 items 字段，则提取列表项
            if ("items" in block) return block.items;

            // 如果是图片，则提取 alt 作为文本说明
            if (block.type === "image") return [block.alt ?? ""];
        })
}

```

```

        // 兜底返回空数组
        return [];
    })
    .join("\n");
}

// 给文章自动设置封面图
function withCoverImage(post: SeedPost): SeedPost {
    return {
        // 保留原文档的所有字段
        ...post,

        // 如果文章没有手动设置封面，则按 uuid 自动生成封面路径
        coverImage: post.coverImage || postCover(post.uuid),
    };
}

// 写入或更新文章数据
function upsertPost(db: SeedDatabase, rawPost: SeedPost) {
    // 先补全封面图
    const post = withCoverImage(rawPost);

    // 执行数据库写入
    run(
        db,
        `
        INSERT INTO post
            (uuid, title, content, content_text, cover_image, status, tags, slug,
            excerpt, view_count, pinned_at, published_at, deleted_at, created_at, updated_at)
        VALUES
            (?, ?, ?, ?, ?, 'published', ?, ?, ?, ?, ?, ?, NULL, ?, ?)
        ON CONFLICT(slug) DO UPDATE SET
            title = excluded.title,
            content = excluded.content,
            content_text = excluded.content_text,
            cover_image = excluded.cover_image,
            status = excluded.status,
            tags = excluded.tags,

```



```

excerpt = excluded.excerpt,
view_count = excluded.view_count,
pinned_at = excluded.pinned_at,
published_at = excluded.published_at,
deleted_at = NULL,
updated_at = excluded.updated_at
`,
[
  // 文章唯一编号
  post.uuid,

  // 文章标题
  post.title,

  // 富文本正文 JSON
  json(buildContent(post.contentBlocks)),

  // 纯文本正文
  contentText(post.contentBlocks),

  // 封面图地址
  post.coverImage,

  // 标签数组转 JSON
  json(post.tags),

  // 文章 slug
  post.slug,

  // 文章摘要
  post.excerpt,

  // 阅读量
  post.viewCount,

  // 如果是置顶文章，则写入置顶时间
  post.pinned ? now : null,

```

```
// 发布时间
post.publishedAt,

// 创建时间使用发布时间
post.publishedAt,

// 更新时间使用当前时间
now,
],
);
}
```

该功能使系统可以通过初始化脚本快速生成完整的课程笔记数据。与手动在后台逐篇录入相比，种子数据方式更适合课程设计项目的初始化、演示和恢复。运行初始化脚本后，数据库中会自动生成“关于博客”和 Vue 3 第 1 章到第 10 章的课程笔记文章，前台小程序即可直接展示这些内容。

5.5 友链展示与技术资料入口功能

友链页用于展示项目技术栈相关的官方资料入口。与传统博客中展示个人友情链接不同，本系统将友链页设计为技术资料导航页，主要展示 GitHub、uni-app、Vue 3、Vue Router、Pinia、TypeScript、Vite、Bun、Elysia、Drizzle ORM、SQLite、Tailwind CSS 等官方链接。这样可以让用户在阅读课程笔记的同时，快速访问相关技术文档。

该功能的实现分为后端数据准备和小程序前端展示两个部分。后端通过友链数据表保存名称、链接地址、头像、描述和标签；前端通过接口获取已通过审核的友链列表，并渲染为卡片。用户点击友链卡片后，可以跳转到对应官方网站。

示例代码如下：

```
// 友链数据示例
const friends = [
  {
    // 友链唯一编号
    uuid: "seed-link-github",

    // 友链名称
    name: "GitHub",

    // 官方链接地址
    url: "https://github.com/",
```

```

// 友链头像地址，可以使用本地上传路径或外部图标地址
avatarUrl: "/api/uploads/friends/github.png",

// 友链描述，说明该网站的主要用途
description:
    "代码托管与协作平台，用来管理项目仓库、Issues、Pull Requests 和开源生态。",

// 标签用于前台展示分类信息
tags: ["代码托管", "开源", "Git"],

// 加入时间，用于后台排序和记录
joinedAt: now - 1 * day,
},
{
    uuid: "seed-link-vue3",
    name: "Vue 3",
    url: "https://vuejs.org/",
    avatarUrl: "/api/uploads/friends/vue3.png",
    description: "渐进式 JavaScript 框架，用于构建现代 Web 用户界面。",
    tags: ["Vue 3", "前端框架", "UI"],
    joinedAt: now - 3 * day,
},
];

// 写入或更新友链
function upsertFriend(db: SeedDatabase, friend: (typeof friends)[number]) {
    run(
        db,
        `
        INSERT INTO friend_link
            (uuid, name, url, avatar_url, description, tags, status, applicant_email,
            reject_reason, joined_at, created_at, updated_at)
        VALUES
            (?, ?, ?, ?, ?, ?, 'approved', NULL, NULL, ?, ?, ?)
        ON CONFLICT(uuid) DO UPDATE SET
            name = excluded.name,
            url = excluded.url,
            avatar_url = excluded.avatar_url,

```

```
description = excluded.description,
tags = excluded.tags,
status = 'approved',
applicant_email = NULL,
reject_reason = NULL,
joined_at = excluded.joined_at,
updated_at = excluded.updated_at
`,
[
// 友链唯一编号
friend.uuid,

// 友链名称
friend.name,

// 友链地址
friend.url,

// 友链头像
friend.avatarUrl,

// 友链描述
friend.description,

// 标签数组转 JSON 后保存
json(friend.tags),

// 加入时间
friend.joinedAt,

// 创建时间
friend.joinedAt,

// 更新时间
now,
],
);
}
```

```
// 初始化友链
export function seedFriends(db: SeedDatabase) {
  // 清空旧友链，避免旧演示数据残留
  run(db, "DELETE FROM friend_link");

  // 逐条写入新的技术栈官方链接
  for (const friend of friends) upsertFriend(db, friend);

  // 返回写入数量，方便脚本输出日志
  return friends.length;
}
```

该功能实现后，友链页不再只是普通外链列表，而是成为项目技术栈资料入口。用户可以通过友链页访问官方文档，管理员也可以在后台继续维护这些链接信息。

5.6 系统配置与个性化信息功能

系统配置功能用于维护站点标题、站点图标、首页横幅、管理员头像、个人简介、社交链接和页脚链接等内容。这些信息保存在 `config` 表中，通过 `config_key` 区分不同配置项。例如 `site_info` 保存站点基础信息，`personal_info` 保存博主个性化信息，`appearance` 保存外观配置，`smtp` 保存邮件服务器配置。

该功能的优点是灵活性较高，不需要为每一种配置单独创建数据库表。配置内容使用 JSON 形式保存，当系统需要新增配置字段时，只需要在配置对象中增加字段即可。

关键代码如下：

```
// 写入或更新配置项
function upsertConfig(
  db: SeedDatabase,

  // 配置键名，例如 site_info、personal_info、appearance、smtp
  configKey: string,

  // 配置值，可以是对象、数组、字符串等
  configValue: unknown,

  // 配置说明，方便后台理解该配置用途
  description: string,

  // 是否公开，1 表示前台可以读取，0 表示仅后台使用
```

```

        isPublic = 1,
    ) {
        run(
            db,
            `
                INSERT INTO config
                (uuid, config_key, config_value, description, is_public, created_at,
updated_at)
                VALUES
                (?, ?, ?, ?, ?, ?, ?)
            ON CONFLICT(config_key) DO UPDATE SET
                config_value = excluded.config_value,
                description = excluded.description,
                is_public = excluded.is_public,
                updated_at = excluded.updated_at
            `,
            [
                // 根据 configKey 生成稳定 uuid
                `seed-config-${configKey}`,

                // 配置键名
                configKey,

                // 配置值转 JSON 字符串保存
                json(configValue),

                // 配置说明
                description,

                // 是否公开
                isPublic,

                // 创建时间
                now,

                // 更新时间
                now,
            ],

```

```
);  
}  
  
// 初始化系统配置  
export function seedConfigs(db: SeedDatabase) {  
  // 初始化外观配置  
  upsertConfig(  
    db,  
    "appearance",  
    {  
      // 当前主题为浅色  
      theme: "light",  
  
      // 主色相, 208 对应偏蓝色调  
      hue: 208,  
  
      // 默认语言为中文  
      language: "zh-CN",  
    },  
    "界面外观配置",  
  );  
  
  // 初始化站点基础信息  
  upsertConfig(  
    db,  
    "site_info",  
    {  
      // 站点标题  
      title: "Blog",  
  
      // 站点图标路径  
      favicon: "/api/uploads/system/favicon.ico",  
  
      // 页脚说明  
      footer: "Blog · Built with uni-app, Vue 3, Bun, Elysia and SQLite",  
  
      // 备案号, 可为空  
      icp: "",  
    },  
  );  
}
```

```

// 页脚技术链接
footerLinks: [
  { name: "GitHub", url: "https://github.com/" },
  { name: "uni-app", url: "https://uniapp.dcloud.net.cn/" },
  { name: "Vue 3", url: "https://vuejs.org/" },
  { name: "Vue Router", url: "https://router.vuejs.org/" },
  { name: "Pinia", url: "https://pinia.vuejs.org/" },
],
},
"站点基础信息",
);

// 初始化个人信息
upsertConfig(
  db,
  "personal_info",
  {
    // 博主用户名
    username: "Sherry",

    // 博主头像
    avatar: "/api/uploads/system/avatar.webp",

    // 个人简介
    bio: "私の 0721 を見て下さいっ",

    // 社交链接
    socialLinks: [
      {
        name: "GitHub",
        url: "https://github.com/NeneAyachi0721",
        iconDark: "/api/uploads/social/GitHub-dark.png",
        iconLight: "/api/uploads/social/GitHub-light.png",
      },
      {
        name: "Bilibili",
        url: "https://space.bilibili.com/436646687",
      },
    ],
  },
);

```



```

        iconDark: "/api/uploads/social/Bilibili-dark.png",
        iconLight: "/api/uploads/social/Bilibili-light.png",
    },
    {
        name: "Steam",
        url: "https://steamcommunity.com/profiles/76561199568695173",
        iconDark: "/api/uploads/social/Steam-dark.png",
        iconLight: "/api/uploads/social/Steam-light.png",
    },
    {
        name: "Email",
        url: "mailto:Sherry1318476070@gmail.com",
        iconDark: "/api/uploads/social/Email-dark.png",
        iconLight: "/api/uploads/social/Email-light.png",
    },
],

// 首页横幅图
hero: "/api/uploads/system/hero.webp",

// 首页主标题
heroTitle: "Blog",

// 首页副标题列表
heroSubtitles: ["Ciallo~(∠・ω< )∩☆"],
},
"博主个性化信息",
);
}

```

通过该配置功能，系统的站点图标、头像、横幅和社交链接等都可以统一管理。管理员既可以通过后台界面修改配置，也可以通过初始化脚本设置默认配置。需要注意的是，配置中保存的是图片路径，真实图片文件仍然需要放置在后端上传目录中，例如 backend/data/uploads/system/ 或 backend/data/uploads/social/。

6. 系统测试

6.1 系统测试概述

系统测试阶段主要验证小程序前台、管理后台、后端接口和数据库初始化是否符合预期。测试方法包括功能测试、接口测试、构建测试、边界输入测试和部署流程测试。

系统测试是软件开发过程中的重要环节，主要目的是验证系统功能是否符合需求分析中的设计目标，检查系统在实际运行过程中是否能够稳定、正确地完成各项业务操作。本系统由微信小程序前台、后台管理端、后端接口服务和 SQLite 数据库组成，涉及文章展示、文章详情、归档浏览、友链展示、管理员登录、文章管理、图片上传、系统配置、邮件日志等多个功能模块。因此，在测试过程中需要从前台展示、后台管理、接口响应和数据持久化等多个角度进行验证。

本系统测试主要采用黑盒测试和功能测试相结合的方式。黑盒测试主要关注用户输入和系统输出是否符合预期，不直接关注程序内部实现过程；功能测试主要针对系统各模块的核心功能进行验证，例如小程序首页是否能够正常加载文章列表，文章详情页是否能够正确显示标题、封面和正文，后台文章管理是否能够保存文章内容，图片上传后是否能够正常显示等。同时，在开发和调试过程中，也结合浏览器控制台、微信开发者工具控制台、后端终端日志和数据库数据检查，对接口请求、错误信息和数据库写入情况进行了辅助验证。

测试环境主要包括 Windows 开发环境、微信开发者工具、浏览器和本地后端服务。测试时需要先启动后端服务，再分别运行后台管理端和小程序前台。后台管理端通过浏览器访问，小程序端通过微信开发者工具进行预览和真机调试。由于小程序真机调试时不能直接访问电脑上的 localhost，因此测试时需要将接口地址配置为电脑在局域网中的 IP 地址，并保证手机和电脑处于同一网络环境下。

本系统测试策略主要包括模块化测试、前后台联动测试、接口测试、数据持久化测试、异常情况测试和兼容性测试。测试过程中，首先按照系统功能模块分别对首页、归档页、文章详情页、友链页、后台登录、文章管理、文件上传和系统配置等功能进行验证，确保各模块能够独立完成预期功能；其次，在后台修改文章、封面、友链和站点配置后，再到小程序前台查看数据是否同步更新，以验证前后台数据的一致性。同时，通过浏览器地址栏、微信开发者工具控制台和页面请求结果检查后端接口是否能够正确返回数据，并验证接口状态码、返回结构和错误提示是否正常。在数据持久化方面，通过新增或修改数据后刷新页面、重新进入系统的方式，检查 SQLite 数据库是否能够正确保存系统数据。此外，还对后端服务未启动、接口地址错误、文章 slug 不存在、上传文件过大、表单字段为空等异常情况进行测试，观察系统是否能够给出合理提示，避免页面空白或程序崩溃。最后，分别在浏览器端、微信开发者工具和真机环境中测试系统运行效果，检查页面布局、数据加载和交互操作是否正常。

通过以上测试方法和策略，可以较全面地验证系统功能的完整性、稳定性和可用性，为系统最终运行提供保障。

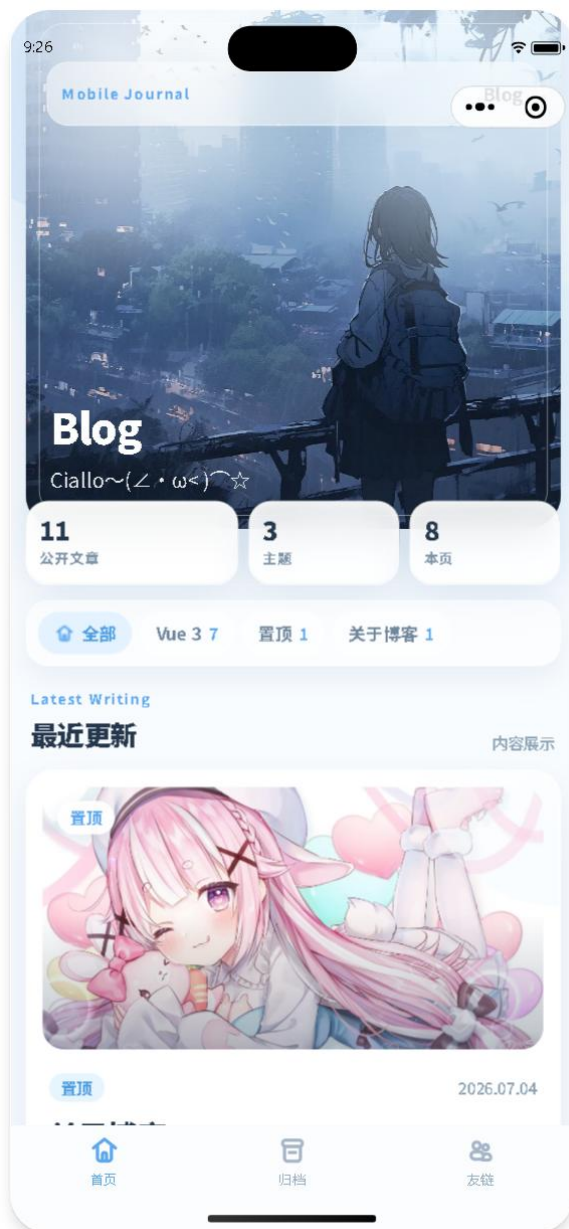
6.2 测试用例

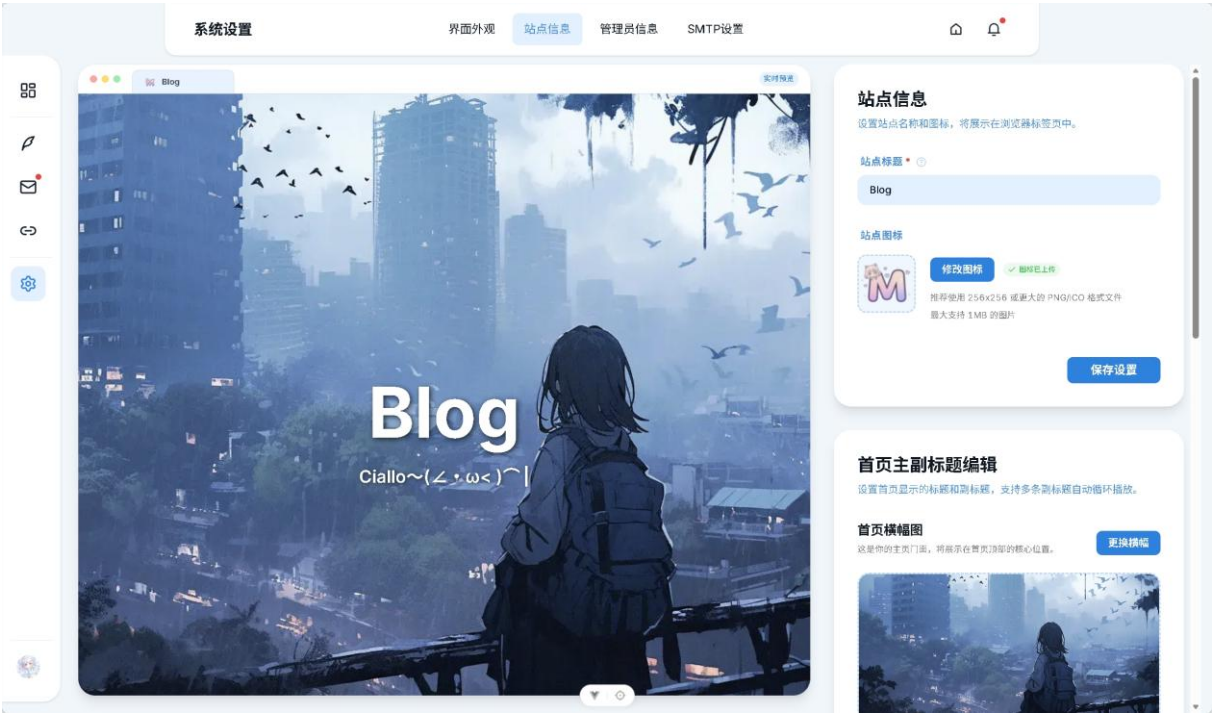
6.2.1 小程序首页模块测试

小程序首页是普通用户进入系统后的主要页面，主要用于展示站点信息、置顶文章、最新文章列表和文章卡片。该模块测试重点是验证首页是否能够正常请求后端数据、文章列表是否正确显示、点击文章是否能够进入详情页，以及页面在加载失败时是否能够显示错误提示。

用例编号	测试用例描述	操作过程	预期结果	实际测试结果
TC001	首页正常加载测试	启动后端服务,打开微信开发者工具,进入小程序首页	首页能够正常显示站点标题、横幅信息和文章列表	通过,首页数据能够正常显示
TC002	最新文章列表显示测试	在后台确认存在已发布文章后,重新打开首页	首页显示已发布文章标题、摘要、标签、封面和阅读量等信息	通过,文章卡片显示正常
TC003	置顶文章显示测试	后台设置文章为置顶状态,刷新小程序首页	置顶文章优先显示在首页醒目位置	通过,置顶文章显示符合预期
TC004	首页文章点击跳转测试	点击首页任意文章卡片	页面跳转到对应文章详情页,并显示文章完整内容	通过,点击后可进入文章详情页
TC005	首页接口异常测试	关闭后端服务后刷新首页	页面不应崩溃,应显示加载失败或空状态提示	通过,页面显示错误提示

系统界面截图：





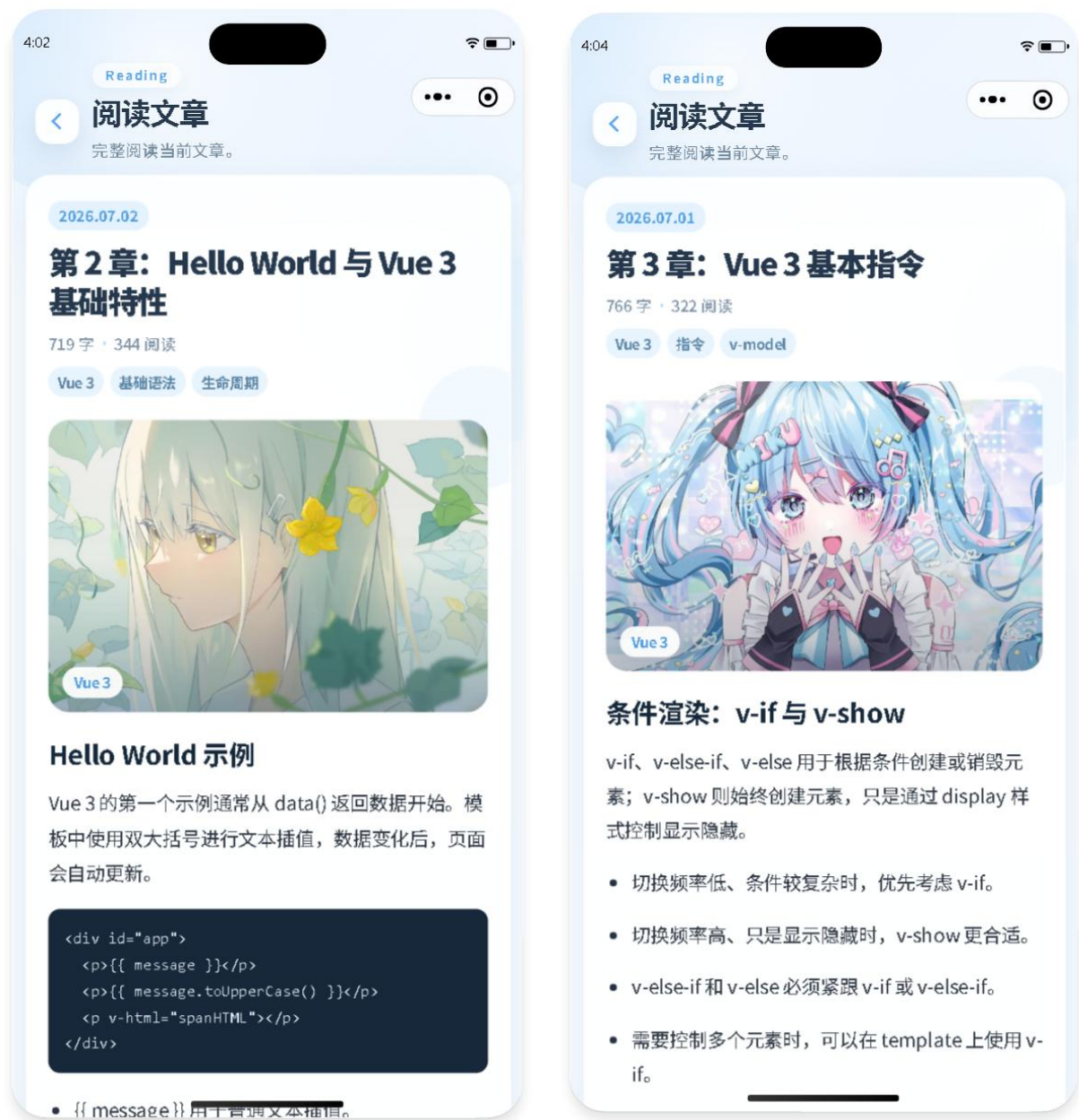
6.2.2 文章详情模块测试

文章详情模块用于展示文章完整内容，是小程序前台的核心阅读页面。该模块测试重点是验证文章详情接口是否正常、文章标题和正文是否显示完整、封面图是否加载、标签和阅读量是否显示、返回按钮是否可用。

用例编号	测试用例描述	操作过程	预期结果	实际测试结果
TC006	文章详情正常加载测试	从首页点击一篇文章进入详情页	页面显示文章标题、发布时间、字数、阅读量、标签和正文内容	通过,文章详情加载正常
TC007	文章封面显示测试	进入设置了封面图的文章详情页	文章封面图显示在正文开头,图片比例正常	通过,封面图显示正常
TC008	富文本正文渲染测试	打开包含标题、段落、列表、代码块和图片的文章	正文内容结构清晰,富文本节点能够正常显示	通过,正文渲染符合预期
TC009	文章标签显示测试	打开带有多个标签的文章详情页	页面显示文章对应标签,标签内容与后台数据一致	通过,标签显示正确
TC010	详情页返回测试	从首页进入详情页后点击返回按钮	页面返回到来源页面,不出现空白页或错误页	通过,返回功能正常

TC011	无效 slug 测试	手动访问不存在的文章 slug	页面显示文章加载失败提示	通过, 异常提示正常
-------	------------	-----------------	--------------	------------

系统界面截图:





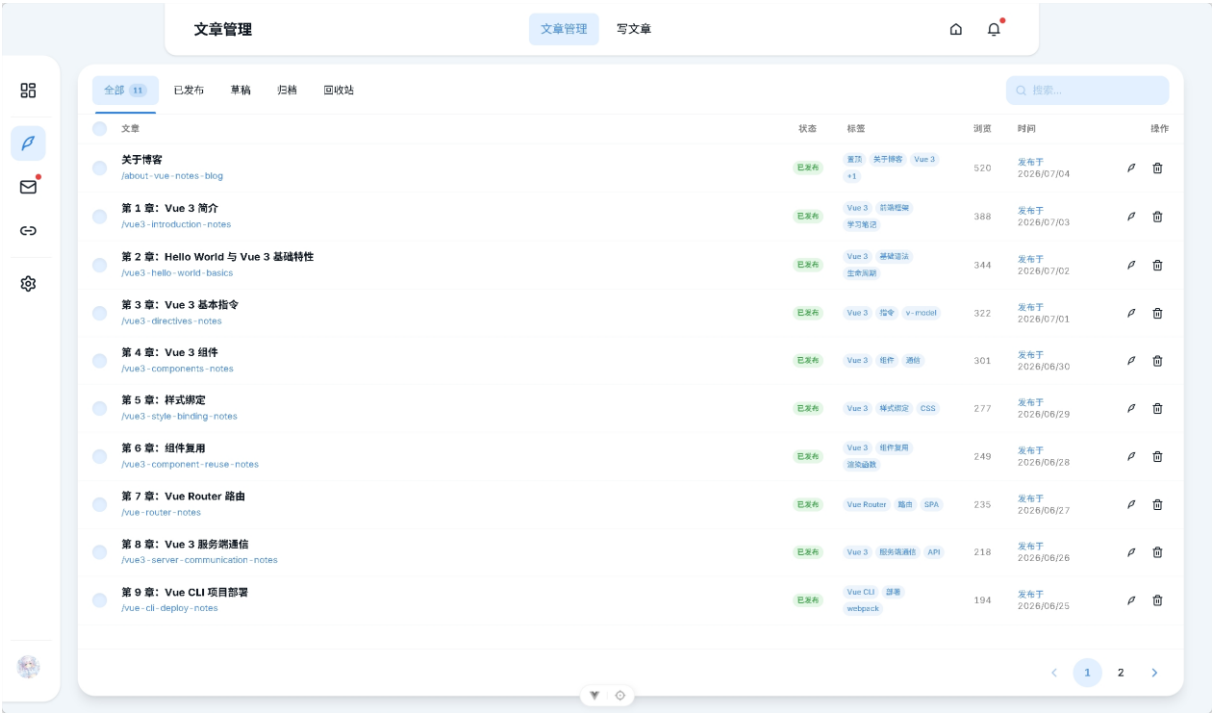
6. 2. 3 归档模块测试

归档模块用于按照时间顺序展示所有已发布文章，方便用户快速浏览课程笔记章节。该模块测试重点是验证归档列表是否完整、文章是否按时间排序、点击归档文章是否能够进入详情页。

用例编号	测试用例描述	操作过程	预期结果	实际测试结果
TC012	归档页正常加载测试	点击底部导航栏“归档”进入归档页	页面能够加载全部公开文章归档数据	通过, 归档数据正常显示
TC013	归档文章排序测试	查看归档页文章顺序	文章按照发布时间进行排序或分组展示	通过, 文章排序符合预期
TC014	归档文章点击测试	点击归档页中的文章条目	页面跳转到对应文章详情页	通过, 跳转正常
TC015	归档空数据测试	将文章状态改为非发布后访问归档页	页面显示空状态提示, 不出现异常	通过, 空状态显示正常

系统界面截图：





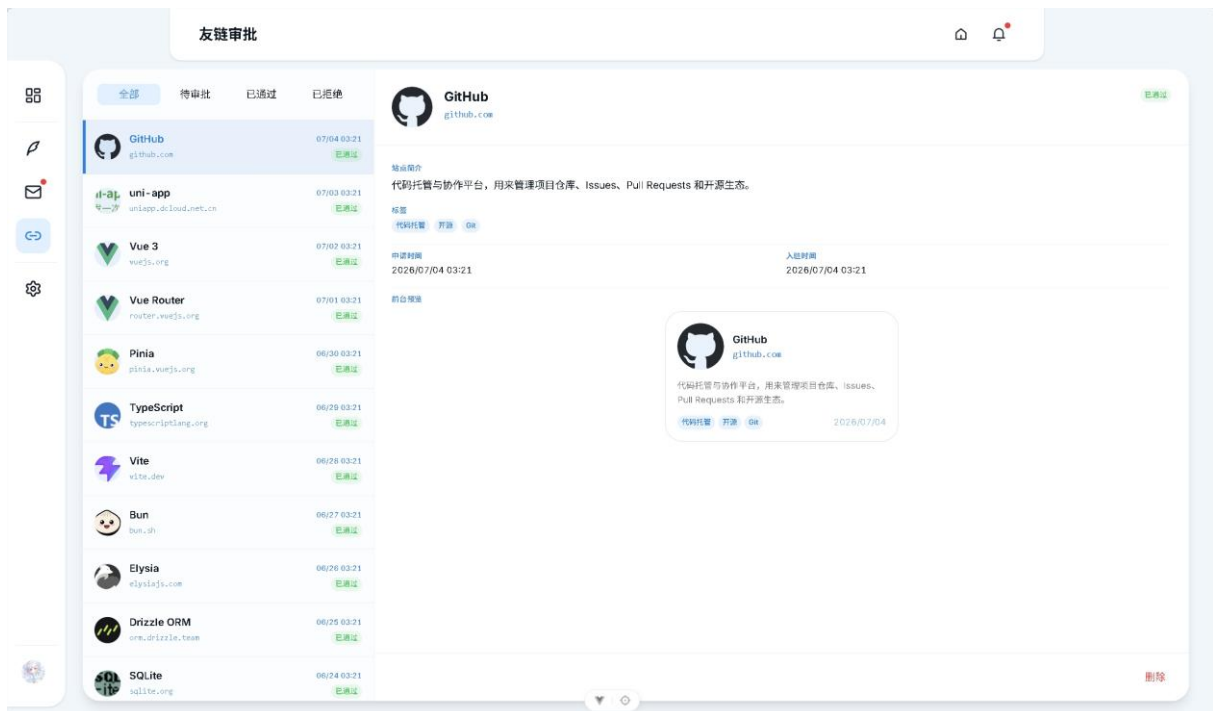
6. 2. 4 友链模块测试

友链模块用于展示项目技术栈相关的官方资料入口，包括 Vue 3、uni-app、GitHub、TypeScript、Vite、Bun、Elysia、SQLite 等链接。该模块测试重点是验证友链数据是否正常加载、图标和描述是否显示、点击链接是否能够跳转。

用例编号	测试用例描述	操作过程	预期结果	实际测试结果
TC016	友链页正常加载测试	点击底部导航栏“友链”进入友链页	页面显示已通过审核的友链列表	通过, 友链列表显示正常
TC017	友链信息显示测试	查看友链卡片内容	每个友链显示名称、头像、描述和标签	通过, 友链信息显示完整
TC018	友链跳转测试	点击某个友链卡片或链接按钮	能够打开对应官方网站或复制链接	通过, 跳转功能正常
TC019	友链图标加载测试	查看友链头像或图标显示情况	图标能够正常显示, 不出现破图	通过, 图标显示正常
TC020	无友链数据测试	清空友链数据后进入友链页	页面显示暂无友链或空状态提示	通过, 空状态显示正常

系统界面截图：





6.3 系统测试总结

经过对系统各个模块的测试，本系统的主要功能均能够正常运行。小程序前台能够完成首页展示、文章详情阅读、归档浏览和友链展示等功能；后台管理端能够完成管理员登录、文章管理、封面上传、友链管理、系统配置和邮件日志查看等功能；后端服务能够正确响应公开接口和管理接口，数据库能够保存文章、配置、友链、用户和邮件日志等数据。

在测试过程中，首先对小程序前台进行了功能验证。测试结果表明，首页能够正常加载站点信息和文章列表，文章详情页能够正确展示标题、发布时间、标签、封面和正文内容，归档页能够按时间展示公开文章，友链页能够显示技术资料入口。页面在后端服务异常或数据为空时，也能够显示相应提示，没有出现页面崩溃或严重显示异常。

其次，对后台管理端进行了测试。测试结果表明，管理员可以正常登录后台，文章可以创建、编辑、保存、发布和删除，文章封面可以上传并在前台显示，系统配置可以保存并同步到前台页面。友链管理和邮件日志管理功能也能够满足基本使用需求，后台整体操作流程较为完整。

再次，对后端接口和数据库进行了测试。测试结果表明，后端服务启动正常，公开文章接口、文章详情接口、友链接口和配置接口均能够返回正确数据。后台修改后的文章、配置和图片路径能够保存到 SQLite 数据库中，服务重启后数据仍然存在，说明系统的数据持久化功能正常。数据库初始化脚本能够生成默认管理员、课程笔记文章、友链和站点配置，便于系统演示和恢复。

在真机调试过程中，发现小程序真机无法直接访问电脑的 `localhost` 地址。经过排查后，将前端接口地址改为电脑局域网 IP，并保证手机和电脑处于同一网络环境，问题得到解决。该问题说明在小程序真机测试时，接口地址配置和网络环境非常重要，也是移动端项目调试过程中需要重点关注的内容。

综合测试结果来看，本系统已经基本达到需求分析阶段提出的设计目标。系统功能较完整，前后台数据能够联动，主要页面显示正常，接口响应稳定，数据库保存可靠。虽然系统仍有进一步优化空间，例如可以增加更完善的搜索功能、标签筛选功能、上传进度显示、错误日志记录和正式部署环境适配等，但从课程设计和实验项目角度来看，系统已经能够满足博客小程序的基本使用需求，具备较好的实用性和可维护性。

7. 总结体会

通过本次博客小程序系统的设计与实现，我对前后端分离项目的开发流程有了更加完整和深入的理解。本系统从最初的需求分析、页面设计，到后来的数据库设计、接口开发、前台展示、后台管理、图片上传和真机调试，基本覆盖了一个完整 Web 项目和小程序项目的主要开发环节。在完成项目的过程中，我不仅巩固了 `Vue 3`、`uni-app`、`TypeScript`、`Bun`、`Elysia`、`SQLite` 等技术的使用方法，也更加清楚地认识到系统开发并不是单纯编写代码，而是需要从功能需求、用户体验、数据结构、接口设计、运行环境和后期维护等多个方面综合考虑。

在前台小程序开发过程中，我体会较深的是移动端页面设计与普通网页设计之间的差异。小程序端屏幕空间有限，页面结构必须更加清晰，文字、卡片、图片、按钮和底部导航之间都需要保持合适的间距。如果页面元素过于拥挤，会影响阅读体验；如果视觉层次不够明确，用户很难快速找到主要内容。因此，在首页、归档页、友链页和文章详情页的实现中，我对页面布局、配色、图标和卡片样式进行了多次调整，使系统整体风格更加统一，也更加适合移动端阅读场景。通过这一过程，我认识到前端开发不仅要实现功能，还要重视用户的实际使用感受。

在后台管理端和后端接口开发过程中，我进一步理解了数据管理的重要性。文章、友链、配置、用户、邮件日志等数据都需要合理存储，并通过接口提供给不同端使用。后台管理端负责修改数据，小程序前台负责展示公开数据，后端则需要在二者之间完成数据校验、权限控制和数据库读写。如果接口设计不清晰，或者数据库字段设计不合理，后期功能扩展和问题排查都会变得困难。因此，在项目实现过程中，我逐渐意识到良好的模块划分、清晰的接口结构和合理的数据库设计，是系统能够稳定运行的重要基础。

本次项目中也遇到了一些实际问题。例如，小程序真机调试时不能直接访问电脑的 `localhost`，需要将接口地址改为电脑在局域网中的 IP；后台上传图片后，数据库中保存的只是图片路径，真实图片文件还需要存在于后端上传目录中；执行数据库初始化脚本时，可能会覆盖后台已经修改过的数据。这些问题在单纯学习语法时并不明显，但在真

实项目开发中却非常常见。通过不断排查和解决这些问题，我对本地开发环境、接口访问、文件上传、数据库初始化和前后端联调有了更实际的认识。

通过本次课程设计，我也认识到测试工作的重要性。系统功能完成后，并不代表项目已经真正可用，还需要对首页加载、文章详情、归档列表、友链展示、后台登录、文章保存、图片上传、系统配置、接口异常和真机调试等功能进行反复测试。测试过程中可以发现许多开发时没有注意到的问题，例如数据为空时的页面显示、接口失败时的错误提示、图片路径错误导致的加载失败等。经过测试和修改，系统的稳定性和完整性得到了提高。

总的来说，本次项目让我将课堂中学习到的前端、后端、数据库和小程序开发知识进行了综合运用。通过完成该博客小程序系统，我不仅掌握了一个完整项目的基本开发流程，也提升了分析问题、解决问题和调试项目的能力。虽然系统目前还存在一些可以继续完善的地方，例如标签筛选、全文搜索、权限细分、正式部署、日志监控等功能仍有优化空间，但从课程设计目标来看，系统已经实现了文章展示、内容管理、友链展示、配置维护和数据持久化等核心功能。后续如果继续完善该项目，可以进一步提升系统的实用性和扩展性，使其成为一个更加完整的个人博客与课程笔记管理平台。

8. 参考资料

- [1] 尤雨溪. Vue.js 设计与实现[M]. 北京: 人民邮电出版社, 2022.
- [2] 张鑫旭. CSS 世界[M]. 北京: 人民邮电出版社, 2017.
- [3] 张鑫旭. CSS 新世界[M]. 北京: 人民邮电出版社, 2021.
- [4] 任钢. TypeScript 编程[M]. 北京: 人民邮电出版社, 2021.
- [5] 王珊, 萨师煊. 数据库系统概论: 第 5 版[M]. 北京: 高等教育出版社, 2014.
- [6] Martin Fowler. 企业应用架构模式[M]. 北京: 机械工业出版社, 2010.
- [7] Roger S. Pressman. 软件工程: 实践者的研究方法: 原书第 8 版[M]. 北京: 机械工业出版社, 2016.
- [8] Ian Sommerville. 软件工程: 原书第 10 版[M]. 北京: 机械工业出版社, 2018.
- [9] Robert C. Martin. 代码整洁之道[M]. 北京: 人民邮电出版社, 2010.

课程设计报告评分

项目	得分
格式与排版（20 分）	
需求分析（10 分）	
系统设计（30 分）	
系统实现（20 分）	
系统测试（20 分）	

指导教师意见：

指导教师签字：

2026 年 7 月 5 日